

FPGA によるマインスイーパーソルバの設計と実装

開示情報に基づく制約推論回路と Virtual JTAG 評価系

情報系学部実験・実習成果報告書

2026 年 8 月 23 日

概要

本報告書では、Intel Cyclone IV E FPGA を搭載する MU500 上に実装したマインスイーパー処理系について述べる。本システムは最大 19×19 セルの盤面を Virtual JTAG 経由で受信し、盤面管理回路、開示情報のみを利用する決定論的ソルバ、評価値計算回路および表示回路を協調動作させる。ソルバは局所制約、盤面全体の残り地雷数、制約集合の差分、小規模フロンティアの完全列挙を組み合わせ、確定セルが存在しない場合に限り限定的な確率選択を行う。完全な盤面を保持するゲーム管理回路と、観測済み情報だけを保持するソルバを RTL 上で分離することで、問題のルールを回路構造として保証した。1000 盤の実機評価では 862 盤を完全に開き、平均スコア 0.903849 を得た。本稿ではアルゴリズムだけでなく、FSM、RAM、FIFO、レジスタ、ハンドシェイクおよび主要な配線を Verilog RTL に対応付けて説明する。

目次

概要	i
第 1 章 はじめに	1
1.1 目的と背景	1
1.2 作成したシステム	1
1.3 本レポートの読み方	2
第 2 章 背景知識	3
2.1 FPGA と RTL	3
2.2 クロック, リセット, clock enable	3
2.3 ready/valid ハンドシェイク	3
2.4 固定ストライドのアドレス計算	4
2.5 同期 RAM とビットマップ	4
2.6 マインスイーパーの制約	4
第 3 章 要求仕様と設計方針	5
3.1 機能要求	5
3.2 インターフェース要求	5
3.3 通信から実行までの要求	5
3.4 非機能要求	5
3.5 モジュール分割の方針	6
3.6 設計上のトレードオフ	6
第 4 章 システム全体構成	7
4.1 トップ階層	7
4.2 決定論的システム境界	7
4.3 盤面ストリームの座標変換	7
4.4 信号の一回の処理	7
4.5 観測可能性の境界	8
4.6 リセットと実行許可	8
第 5 章 盤面管理回路: minesweeper_game_core	9
5.1 役割と情報の所有権	9
5.2 主要レジスタとメモリ	9

5.3	FSM	9
5.4	選択の受理と安全性検査	10
5.5	REVEAL とイベント配線	10
5.6	0 セルの幅優先探索	10
5.7	実装上の注意	10
第 6 章	決定論的ソルバのアルゴリズムと RTL 実装	12
6.1	ソルバの観測モデルとセル状態	12
6.1.1	セル状態の符号化	13
6.1.2	固定ストライドによるアドレス	13
6.2	全体 FSM と 1 クロック処理	13
6.2.1	開始とクリア	13
6.2.2	選択と開示の 1 エントリ・パイプライン	15
6.3	局所推論 R1–R3	15
6.4	安全・変更・dirty キューのハードウェア	16
6.5	グローバル推論 R4	17
6.6	制約キャッシュと R5 部分集合差分	17
6.6.1	5×5 共通座標への変換	17
6.6.2	派生制約ストア	18
6.6.3	R5 の親 FSM 配線	18
6.7	ハードウェアとしての不変条件とエラー処理	18
第 7 章	制約コンポーネントの構築と全列挙回路	20
7.1	成分処理の全体配線	20
7.2	局所マスクからセル番号への変換	20
7.3	コンポーネント・ビルダの内部レジスタ	21
7.4	小成分パイプラインの FSM	22
7.5	DFS 列挙器のアルゴリズム	23
7.5.1	内部 RAM と集計値	23
7.5.2	状態遷移と枝刈り	23
7.5.3	列挙結果からの強制判定	23
7.6	強制セル walker と親ソルバへの書込み	24
7.7	成分の収集と重複抑制	25
7.8	確率候補の生成と比較回路	25
7.9	回路資源とアルゴリズムの対応	26
第 8 章	行き詰まり時の選択戦略と確率フィードバック	27
8.1	行き詰まりの定義と制御フロー	27
8.2	角選択	28
8.3	成分候補の選択	29
8.3.1	候補の意味	29

8.3.2	候補探索のレジスタと状態	29
8.4	候補回数の上限と半数条件	29
8.5	辺中央救済選択	30
8.6	推測とスコアの関係	30
8.7	戦略のハードウェア上のトレードオフ	31
第 9 章	JTAG 通信とクロックドメイン交差	32
9.1	通信の役割	32
9.2	64 ビットコマンド形式	32
9.3	TCK 側のシフタ	33
9.4	CDC メールボックス	33
9.5	プロトコルエンジンとバックプレッシャ	33
第 10 章	評価値計算と表示回路	35
10.1	スコアの定義と固定小数点化	35
10.2	メトリクス FSM	35
10.3	逐次除算器	35
10.4	7 セグメント表示	36
10.5	速度制御	36
第 11 章	シミュレーション, 実機試験, ビルド設定	37
11.1	RTL シミュレーション	37
11.2	Virtual JTAG 実機試験	37
11.3	Quartus プロジェクト	38
11.4	検証項目	38
第 12 章	実機評価と考察	39
12.1	speed10 の結果	39
12.2	スコア分布	39
12.3	サイクル数の解釈	39
12.4	stalled の意味	40
12.5	考察	40
第 13 章	回路実装上の工夫	41
13.1	固定サイズメモリ	41
13.2	逐次化とハンドシェイク	41
13.3	キューと bitmap	41
13.4	全列挙の上限と枝刈り	41
13.5	除算を避ける比較	42
13.6	観測面と完全盤面の分離	42
13.7	クロックと通信の安全性	42

第 14 章	制約，課題，今後の改善	43
14.1	アルゴリズム上の制約	43
14.2	ヒューリスティックの影響	43
14.3	固定容量の課題	43
14.4	通信・性能の課題	43
14.5	評価の拡張	44
14.6	保守性	44
第 15 章	まとめ	45
第 A 章	モジュール一覧	46
A.1	提出構成と比較用構成	47
第 B 章	ビルドおよび実行手順	48
B.1	RTL シミュレーション	48
B.2	Quartus コンパイル	48
B.3	実機書込みと 1000 盤試験	48
B.4	本文の生成	49
参考文献		50

目次

1.1	FPGA マインスイーパーの処理経路	1
2.1	ready/valid の転送成立例（両信号が 1 のクロックで成立）	4
3.1	機能分割と境界信号	6
4.1	board_stream_controller の状態遷移	8
5.1	ゲームコアの開示・連鎖 FSM	10
6.1	ゲームコアとソルバの情報境界	12
6.2	決定論的ソルバの上位 FSM（詳細状態をグループ化）	14
6.3	開示イベントを同期 RAM へ反映する概念タイミング	15
6.4	solver_neighbor_scan の FSM	16
6.5	変更伝搬と安全セル選択のキュー配線	16
6.6	R5 のキャッシュ, 比較器, 派生制約ストア	17
7.1	小規模制約コンポーネントの処理パイプライン	20
7.2	派生制約の 5×5 マスクを疎なセル列へ変換するイメージ	21
7.3	小成分パイプラインの状態遷移	22
7.4	DFS における上限・下限枝刈りの概念例	24
7.5	確率候補を交差積で比較するデータパス	25
8.1	論理推論が止まった後の選択制御	28
8.2	角選択の順序（初手を含む）	28
8.3	安全セル半数条件の組合せデータパス	30
8.4	辺中央救済の 4 候補	31
9.1	JTAG から盤面ストリームへの信号経路	32
9.2	64 ビットワードの概念図（フィールド幅は opcode ごとに再利用される）	32
10.1	スコア計算のデータパス	36
11.1	公式ファイルシミュレーションの検証経路	37
12.1	スコア分布の描画例（最終版では CSV から自動生成する）	40

表目次

6.1	knowledge_mem のセル状態	13
6.2	トップ・ソルバの主な状態レジスタと役割	14
6.3	ソルバの主な不変条件	19
7.1	solver_component_builder のデータ構造	21
7.2	solver_small_enumerator の DFS 状態	24
7.3	数学的対象と RTL 要素の対応	26
8.1	提出トップの行き詰まり対策パラメータ	27
8.2	確率候補に関するレジスタ	29
9.1	JTAG opcode と応答	33
12.1	speed10 実機 1,000 盤の集計	39
A.1	RTL モジュールと主な役割	46

第 1 章

はじめに

1.1 目的と背景

本実験では、マインスイーパの盤面管理と自動解答アルゴリズムを FPGA 上の RTL 回路として実装した。マインスイーパは、開いたマスに表示される数字を手掛かりに、周囲の地雷位置を推定する問題である。ソフトウェアでは盤面を配列として走査し、関数呼出しや動的データ構造を利用できるが、FPGA では処理をクロックごとの状態遷移と固定容量のメモリに分解しなければならない。本実装の目的は、このアルゴリズムを実際に動作する同期回路へ変換し、通信、盤面管理、推論、評価表示までを一つの処理系として構成することである。

1.2 作成したシステム

提出トップは minesweeper_jtag_submission_local_top であり、20 MHz クロック CLK_20MHZ とリセット RESET_N を受ける。このトップは minesweeper_jtag_top をパラメータ設定して使用し、決定論的ソルバ、JTAG 動作、フルスピード動作、最大 15 回の確率的選択、通常 8 回の確率的選択を有効にしている。盤面の最大値は $19 \times 19 = 361$ セルである。

図 1.1 は、PC から FPGA までの情報の流れを示す概念図である。PC 側の JTAG コマンドは、盤面ヘッダとセル列に分けられ、FPGA 内部で `cfg_valid`, `load_valid`, `start_solver` に変換される。処理結果はスナップショットとして保持され、PC が読み出すまで失われない。

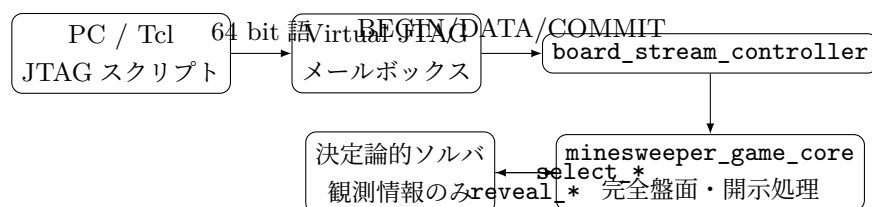


図 1.1: FPGA マインスイーパの処理経路

1.3 本レポートの読み方

第 2 章では FPGA, RTL, クロック同期回路, ready/valid 通信, RAM, FSM などの基礎を説明する。第 3 章で要求仕様と設計方針, 第 4 章でモジュールの接続を示す。第 5 章では完全盤面を管理するゲームコアを回路レベルで説明する。以降では決定論的推論, 制約列挙, JTAG 通信, 評価回路, 検証結果を順に扱う。特に, アルゴリズム上の集合やキューが, どのレジスタ, RAM, FIFO, および信号線で実現されているかを対応づけて記述する。

第 2 章

背景知識

2.1 FPGA と RTL

FPGA (Field Programmable Gate Array) は、論理関数を構成する LUT、状態を保存するフリップフロップ、配線資源、および組込みメモリを再構成できる集積回路である。本報告でいう RTL (Register Transfer Level) は、レジスタ間でどのデータを転送し、クロックのどのタイミングで値を更新するかを記述する設計抽象度である。Verilog の `always @(posedge clk)` は順序回路を表し、非ブロッキング代入 `<=` は同一クロック端で同時にレジスタを更新する。一方、`always @*` と `assign` は現在の入力から出力を求める組合せ回路である。

したがって、ソフトウェアの「while ループ」は、通常、`state` レジスタと条件分岐を持つ FSM (有限状態機械) へ変換される。例えばゲームコアの近傍走査は、一クロックに一つの隣接位置だけを調べ、`scan_dx`, `scan_dy` を更新する。この分割により、クロック周波数、メモリの読み出し遅延、回路面積を制御できる。

2.2 クロック, リセット, clock enable

本システムの主クロックは 20 MHz であり、周期は 50 ns である。通常の RTL ではクロックそのものを論理ゲートで止めず、`run_enable` の条件で状態更新を抑制する。これはクロックのスキューやゲーティングによるタイミング問題を避けるためである。非同期リセット入力は各モジュールの `reset` に接続され、状態、キューのポインタ、エラーラッチを既知値へ戻す。

2.3 ready/valid ハンドシェイク

ストリーム転送では、送信側が `valid=1` にしてデータを提示し、受信側が `ready=1` で受け入れ可能であることを示す。両方が同じクロックで 1 のときに 1 ワードを転送する。この規約は送信側の速度と受信側の処理速度が異なる場合にも使える。ゲームコアでは、設定を `cfg_valid/cfg_ready`, セルを `load_valid/load_ready`, 選択を `select_valid/select_ready` で伝える。

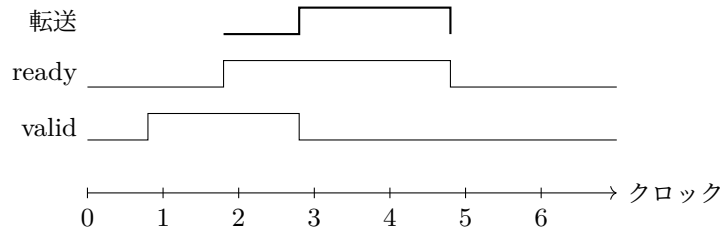


図 2.1: ready/valid の転送成立例（両信号が 1 のクロックで成立）

2.4 固定ストライドのアドレス計算

盤面は二次元座標 (x, y) で表すが, RTL の RAM は一次元配列である. 最大幅を 19 とし, 実際の幅が小さい場合も行間隔を 19 に固定すると, セルアドレスは

$$i = 19y + x = (y \ll 4) + (y \ll 1) + y + x$$

となる. 乗算器を用いずシフトと加算だけで実装できるため, `select_index_calc`, `current_index_calc`, `neighbor_index_calc` と, ストリーム側の `load_index` は同じ規則を使う.

2.5 同期 RAM とビットマップ

同期 RAM ではアドレスをクロック端で受け付け, データを次のクロックで出力する. `solver_state_ram` の `knowledge_mem` はこの方式で読み出される. 一方, 開封済みか, キューへ登録済みかのような 1 ビット情報は `opened`, `scheduled` のビットマップで保持する. ビットマップはセル番号をそのままビット位置に対応させ, 重複登録判定を一クロックの論理で行える.

2.6 マインスイーパの制約

開いた数字セルの値を c , 隣接する既知地雷数を m , 未知隣接セル集合を U とすると, 未知集合に含まれる地雷数は $r = c - m$ である. $r = 0$ なら全て安全, $r = |U|$ なら全て地雷である. この局所制約を回路内で再評価し, 確定したセルを FIFO に送ることがソルバの基本動作となる.

第 3 章

要求仕様と設計方針

3.1 機能要求

本システムは、PC から受け取った一つの盤面を FPGA 内部へロードし、選択と開示を繰り返して結果を返す。幅と高さは 1~19、地雷数は 1 以上かつセル総数未満、セル値は 0~9（9 は地雷）とする。提出トップでは JTAG を唯一の外部制御経路として固定し、盤面入力、処理開始、状態照会、結果読み出し、ACK（結果受領通知）を実現する。

3.2 インターフェース要求

ゲームコアの主要信号は次の通りである。設定は `cfg_width`, `cfg_height`, `cfg_total_mines` と `cfg_valid/cfg_ready` で転送する。セルは `load_index` と `load_value` で転送する。ソルバからゲームコアへの選択は `select_x`, `select_y`, `select_valid/select_ready`, 逆方向の観測情報は `reveal_valid`, `reveal_x`, `reveal_y`, `reveal_is_mine`, `reveal_clue` で表す。この分離により、ソルバは未開示セルの本当の値を入力として受け取らない。

3.3 通信から実行までの要求

`board_stream_controller` は、入力トランザクションを次の FSM で管理する。

`ST_IDLE → ST_CFG → ST_LOAD → ST_COMMIT → ST_RUN`

BEGIN で盤面属性を受け取り、CFG でゲームコアへ設定を送り、DATA を行優先順でロードし、COMMIT で `start_solver` を 1 クロックだけ生成する。実行中に `result_valid` を受けると `board_done` を出し、次の盤面を受け付ける。セル順序の欠落、値が 9 を超えるセル、違法なヘッダは `protocol_error` にラッチする。

3.4 非機能要求

- 20 MHz クロックで合成可能な同期回路であること。
- $19 \times 19 = 361$ セルを固定容量で保持し、動的メモリを使わないこと。
- 完全盤面はゲームコアだけが参照し、ソルバは開示イベントだけを参照すること。

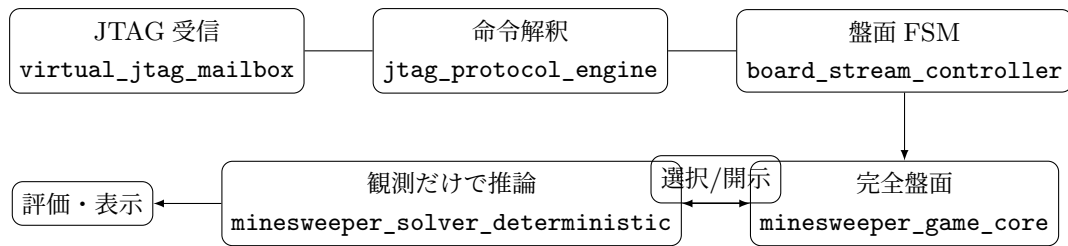


図 3.1: 機能分割と境界信号

- FIFO の空・満杯，範囲外座標，重複開示，制約矛盾を検出して `protocol_error` または `stalled` に反映すること．
- 除算器や浮動小数点演算をアルゴリズムの各クロック経路に置かず，固定小数点または比較可能な整数演算を用いること．

3.5 モジュール分割の方針

処理を通信(`virtual_jtag_mailbox`, `jtag_protocol_engine`), ストリーム変換(`board_stream_controller`), 盤面管理 (`minesweeper_game_core`), 推論 (`minesweeper_solver_deterministic` と下位ブロック), 評価・表示に分割した．各境界はクロック同期のハンドシェイクとし，内部のアルゴリズムを JTAG のビット配置から独立させた．

3.6 設計上のトレードオフ

全 361 セルを毎クロック完全走査すれば単純だが，論理量と処理時間が増える．そこでソルバは `solver_dirty_fifo` を使い，変更されたセルの近傍だけを再評価する．一方，制約の全列挙は変数数を最大 12 に制限し，最大 $2^{12} = 4096$ 通りに抑える．固定上限により回路規模を予測できる反面，大きな連結成分では `stalled` となる可能性がある．

第 4 章

システム全体構成

4.1 トップ階層

提出トップ `minesweeper_jtag_submission_local_top` は、物理ピンと動作パラメータを定める薄いラッパである。その内部の `minesweeper_jtag_top` から、JTAG、ストリーム制御、決定論的システム、表示・メトリクスが接続される。外部には `CLK_20MHZ`, `RESET_N`, `HEX_A/HEX_B` と 9 本の表示選択 `SEG_SEL`, 8 本のセグメントバス `SEG_A`~`SEG_H` がある。

4.2 決定論的システム境界

`minesweeper_deterministic_system` は、ゲームコアとソルバを接続する統合境界である。設定を受理すると `active_width`, `active_height`, `active_mines` レジスタへ保存し、ソルバにはこれらのラッチ値を渡す。ゲーム側の `game_error` とソルバ側の `solver_error` は

$$\text{protocol_error} = \text{game_error} \mid \text{solver_error}$$

として外部へ出力される。選択は `select_valid/select_ready`, 観測は `reveal_valid` と `cascade_done` で同期する。主な結果線は `solver_busy`, `solver_done`, `solver_stalled`, `selections_issued`, `opened_safe_count`, `opened_mine_count`, `cycle_count` である。

4.3 盤面ストリームの座標変換

外部のセル番号 `cell_ordinal` は、実際の盤面幅に対する密な行優先番号である。`board_stream_controller` 内の `load_x`, `load_y` はロードのたびに更新され、ゲームコアへは固定ストライドの `load_index = 19load_y + load_x` を出力する。`expected_ordinal` と `expected_cells` は、セルが飛ばされず、ヘッダで宣言した個数と一致することを監視するレジスタである。

4.4 信号の一回の処理

設定転送が成立するクロックでゲームコアは `width`, `height` を更新し、`configured` を合法性判定結果にする。続くセル転送では `board_mem[load_index]` が更新される。全セルを受け取って

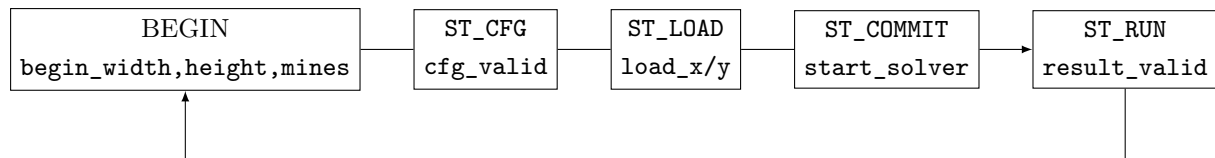


図 4.1: board_stream_controller の状態遷移

COMMIT が成立すると、ソルバが最初の select_x/y を発行する。ゲームコアは選択を受理すると reveal_* を生成し、ソルバはそのイベントを取り込んで次の選択を決める。この繰返しは、ゲームコアから cascade_done が出て選択待ちへ戻るまで続く。

4.5 観測可能性の境界

完全な地雷配置を保持する配列は minesweeper_game_core の board_mem だけである。ソルバ側の solver_state_ram はセル状態と開示された数字だけを保持し、未開示セルの真値は書き込まれない。この構造は単なるソフトウェア上の規約ではなく、モジュールの入出力ポートから強制される情報フローである。

4.6 リセットと実行許可

リセット直後は各 FSM がアイドル状態となり、キューの queue_head, queue_tail, queue_count とエラーラッチがクリアされる。run_enable=0 の間はゲームコアの状態更新を停止し、JTAG 設定などアイドル時に許された処理だけを受ける。提出設定の FORCE_FULL_SPEED=1 ではソルバの有効化を常時許可する。

第 5 章

盤面管理回路：

minesweeper_game_core

5.1 役割と情報の所有権

minesweeper_game_core は盤面の完全な値を所有し，受理されたセル選択を開示イベントへ変換する唯一のモジュールである．入力 load_value=9 は地雷，0～8 は数字セルを表す．ソルバは reveal_is_mine, reveal_clue と座標だけを受け取るため，未開示セルの値を直接参照できない．

5.2 主要レジスタとメモリ

board_mem[0:360] 4ビットの完全盤面 RAM．セル値 9 を地雷として保持する．

opened 開封済みセルの 361 ビットマップ．既開封セルの再選択を拒否する．

scheduled 連鎖展開キューへ登録済みであることを示す 361 ビットマップ．同じセルの二重登録を防ぐ．

queue_mem[0:360] y, x を結合した 10 ビット座標を格納する円環 FIFO．

queue_head, queue_tail, queue_count FIFO の読み出し位置，書き込み位置，要素数．
queue_increment は 360 の次を 0 に戻す．

current_x, current_y 現在 REVEAL する座標．scan_dx, scan_dy は 3×3 近傍の走査位置．

opened_safe_count, opened_mine_count 開いた安全セル数と地雷数．評価回路へ渡す．

5.3 FSM

状態レジスタ state は次の 5 状態を持つ．

1. STATE_IDLE: 設定，セルロード，または選択を ready/valid で受理する．
2. STATE_DEQUEUE: queue_head から座標を読み，current_x/y へ移す．
3. STATE_REVEAL: 開封ビットを立て，盤面值を読み，reveal_* を 1 クロック生成する．
4. STATE_SCAN: 0 セルの周囲 8 セルを一つずつ検査する．
5. STATE_CHECK: FIFO が空なら cascade_done を出して IDLE へ戻り，非空なら DEQUEUE へ戻る．

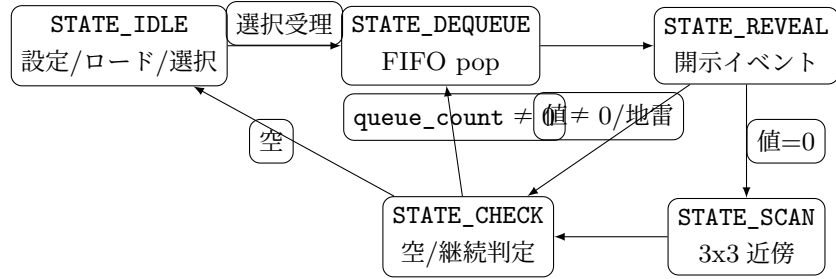


図 5.1: ゲームコアの開示・連鎖 FSM

5.4 選択の受理と安全性検査

IDLE では `select_ready` が `run_enable`, `configured`, IDLE 状態, かつ設定・ロード転送がないことの論理積になる. `select_index_calc` は $(select_y \ll 4) + (select_y \ll 1) + select_y + select_x$ であり, 範囲外座標または `opened[index]` が 1 なら `select_rejected` と `cascade_done` を出す. 受理時は `queue_mem[0]` に座標を書き, `scheduled[index]` を 1 にして DEQUEUE へ遷移する.

5.5 REVEAL とイベント配線

DEQUEUE では FIFO の座標を `current_x/y` にラッチし, REVEAL の次クロックで `board_mem[current_index_calc]` を参照する. 値が 9 なら `reveal_is_mine=1`, `reveal_clue=0` とし, `opened_mine_count` を加算する. 安全セルでは `reveal_is_mine=0`, `reveal_clue=board_mem[...]` とし, `opened_safe_count` を加算する. これらの信号は非同期的な組合せ出力ではなく, クロックに同期した `reveal_valid` イベントである.

5.6 0 セルの幅優先探索

0 セルを開いた場合, `scan_dx` と `scan_dy` を 0 へ戻し SCAN に入る. 各クロックで

$$n_x = \text{current_x} + \text{scan_dx} - 1, \quad n_y = \text{current_y} + \text{scan_dy} - 1$$

を計算し, 中心 ($dx = 1, dy = 1$) を除く 8 近傍を検査する. 範囲内, 未登録 (`!scheduled[n]`), 非地雷 (`board_mem[n] != 9`) なら, 座標を `queue_mem[queue_tail]` に書き, `queue_tail` と `queue_count` を更新する. `scheduled` を先に立てることで, 同じセルが複数の親セルから発見されても一度だけキューに入る. `queue_count==MAX_CELLS` のときは `protocol_error` を立てる.

5.7 実装上の注意

Verilog の配列をソフトウェア配列と同じ感覚で多重に読み出せるとは限らない. 本コアでは一回の FSM 状態で処理する対象を限定し, 座標計算を組合せ信号, 状態・ポインタ更新をクロック同期レジスタへ分けている. また, `width` と `height` は設定時に合法性判定され, 不正な値なら

`configured=0` と `protocol_error=1` になる．この範囲検査と FIFO 満杯検査は，入力データが正しい場合だけでなく，通信不良時に回路が無限に動き続けることを防ぐ．

第 6 章

決定論的ソルバのアルゴリズムと RTL 実装

本章では、`rtl/solver/minesweeper_solver_deterministic.v` (以下、トップ・ソルバ) を中心に、マインスイーパの推論規則をどのように同期回路へ変換したかを説明する。ここでいう「決定論的」とは、開示された数字から論理的に必ず成り立つ結論（安全または地雷）を優先し、それでも結論が得られない場合だけ、後述する限定的な候補選択へ進むという意味である。ソルバは地雷を含む完全盤面を入力として受け取らない。入力されるのは `reveal_valid`, `reveal_x`, `reveal_y`, `reveal_is_mine`, `reveal_clue` という開示イベントだけであり、この境界がアルゴリズム上の観測制約をハードウェアとして保証する。

6.1 ソルバの観測モデルとセル状態

ソフトウェアのマインスイーパ解答器は通常、盤面配列と既知情報を同じメモリに置いてしまう。本設計では、完全盤面を保持する `minesweeper_game_core` と、観測情報だけを保持する `solver_state_ram` を分離した。ゲームコアからソルバへは、選択に応じた開示結果だけが転送される。したがって、ソルバ内部のメモリを調べても、まだ開いていないセルの地雷／安全値は存在しない。

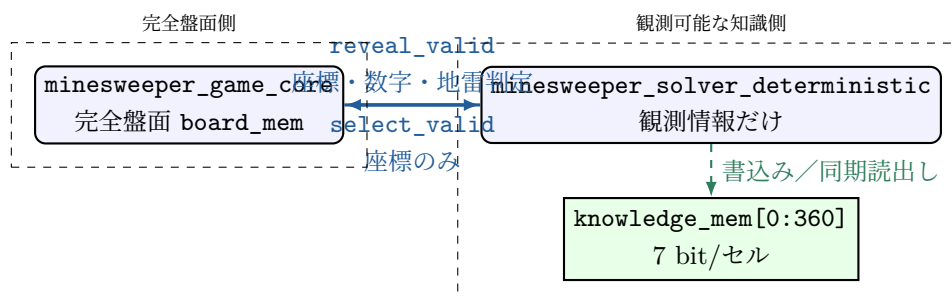


図 6.1: ゲームコアとソルバの情報境界

表 6.1: knowledge_mem のセル状態

符号	Verilog 定数	意味
3'd0	CELL_UNKNOWN	未知セル（まだ開示されていない）
3'd1	CELL_QUEUED_SAFE	安全と推論され、選択 FIFO に入ったセル
3'd2	CELL_OPEN_SAFE	開示済み安全セル（数字を保持）
3'd3	CELL_KNOWN_MINE	論理的に地雷と分かったセル
3'd4	CELL_OPEN_MINE	実際に選択して開いた地雷

6.1.1 セル状態の符号化

solver_state_ram の knowledge_mem は 361 エントリ、1 エントリ 7 bit である。下位 3 bit の状態と上位 4 bit の数字を次のように格納する。

この分類により、例えば CELL_QUEUED_SAFE をもう一度安全キューへ積まない、開示済み安全セルを誤って再選択しない、という検査が行える。セルを地雷と論理判定しても、実際に開くまでは CELL_KNOWN_MINE であり、observed_mine_count には加算されない。

6.1.2 固定ストライドによるアドレス

盤面の有効幅は 1~19 だが、内部アドレスのストライドは常に 19 である。座標 (x, y) は次式で 9 bit のセル番号へ変換する。

$$i = 19y + x = (y \ll 4) + (y \ll 1) + y + x. \quad (6.1)$$

reveal_index_wide, scan_cell_index, active_cell_index などがこの式を使う。定数 19 を乗算器で実現せずシフトと加算に分解するため、合成器が小さな加算回路として実装できる。幅が 10 の場合でも行末の 9 セルは未使用であり、範囲検査は `pending_reveal_x < width, pending_reveal_y < height` で行う。

6.2 全体 FSM と 1 クロック処理

トップ・ソルバは state (6 bit) を状態レジスタとする単一の大きな FSM である。主要状態を図 6.2 に示す。全状態を一度に説明すると読者が見失うため、**開示処理**、**R1-R4 処理**、**R5 / 成分処理**、**推測処理** の 4 段階に分ける。各段階で外部 RAM を読む場合は、アドレス設定状態、読出し待ち状態、データ判定状態を分けている。これは solver_state_ram の同期読出し（アドレスをクロックで登録し、次のクロックで read_data を得る）に対応する。

6.2.1 開始とクリア

S_IDLE で start を受けると、width, height, mines, 各カウンタをラッチし、unknown_count <= board_width * board_height, remaining_mines <= total_mines とする。同時に

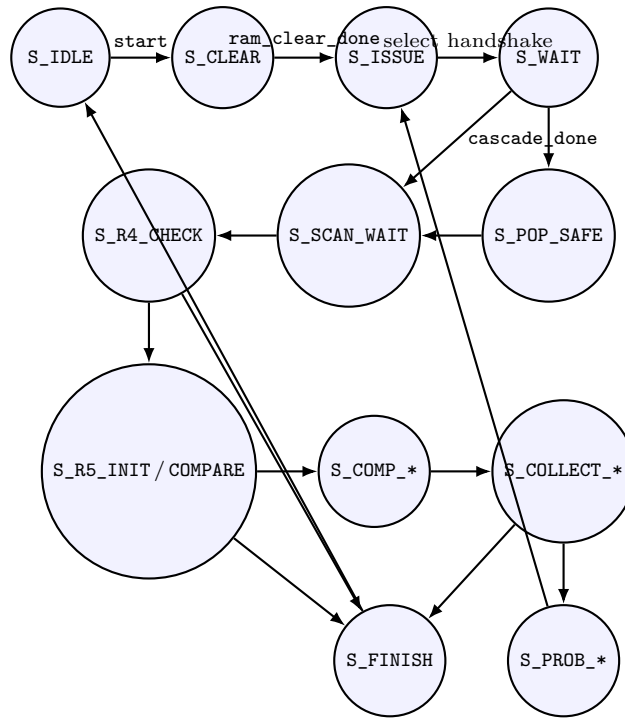


図 6.2: 決定論的ソルバの上位 FSM（詳細状態をグループ化）

表 6.2: トップ・ソルバの主な状態レジスタと役割

レジスタ	幅	役割
state	6 bit	逐次処理の現在状態（S_IDLE～S_RESCUE_CHECK）
width,height,mines	5,5,9 bit	盤面形状と総地雷数のラッチ
active_x,active_y	5,5 bit	次に select_*へ出す座標
scan_x,scan_y,scan_clue	5,5,4 bit	近傍走査／全体走査の中心
unknown_count,remaining_mines	9,9 bit	未知セル数と未確定地雷数
queued_safe_count	9 bit	安全 FIFO に入った未処理セル数
pending_reveal_*	9+5+5+1+4 bit	同期 RAM 検査中の 1 件の開示イベント
constraint_valid	361 bit	各数字セルのキャッシュ制約が有効かを示す bitmap

ram_clear_start と fifo_clear を 1 にする. solver_state_ram は clear_busy 中の clear_index を 0 から 360 まで 1 セルずつ書き換えるため, 361 クロック後に ram_clear_done が出る. 盤面形状が範囲外, 地雷数が 0, または地雷数がセル数以上なら protocol_error を立てて S_ERROR へ遷移する.

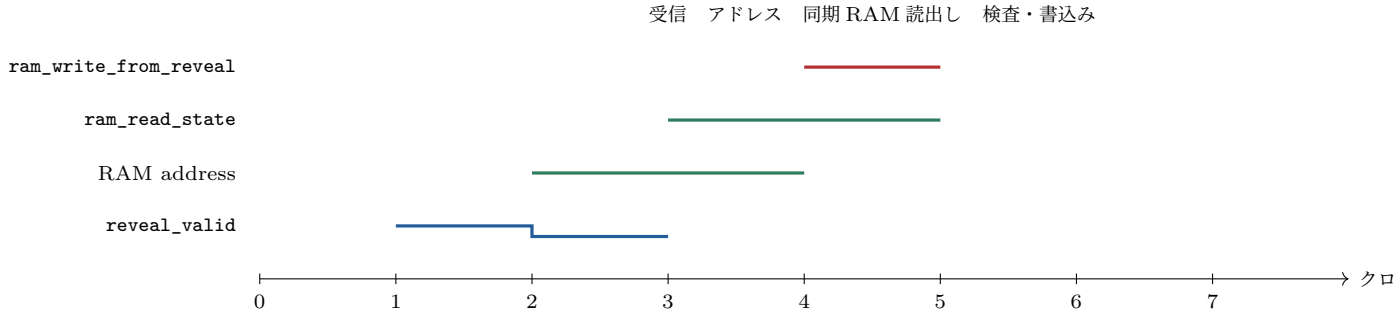


図 6.3: 開示イベントを同期 RAM へ反映する概念タイミング

6.2.2 選択と開示の 1 エントリ・パイプライン

S_ISSUE では `select_valid=1` となり, `select_ready=1` のクロックで `active_x, active_y` をゲームコアへ渡す. その後 S_WAIT で `reveal_valid` を受ける. 開示情報は直接 RAM へ書かず, `pending_reveal_index`, `pending_reveal_is_mine`, `pending_reveal_clue` へ一度保存する. `ram_read_index` にその番号を出し, 次クロックの `ram_read_state` と突き合わせてから `ram_write_from_reveal` を生成する.

`reveal_transition_valid` は, 座標範囲, 手掛かり値 (安全なら 8 以下), 現在状態, カウンタの整合性, 変更 FIFO への投入可能性を同時に検査する. 地雷開示なら `remaining_mines` を減らし, 安全開示なら `observed_safe_count` を増やす. この検査に失敗しても黙って状態を上書きせず, `protocol_error` を立てる.

6.3 局所推論 R1–R3

開示された数字セルを中心 c とし, その 8 近傍のうち未知セル集合を U , 既知地雷数を m , 表示数字を n とする. 制約は

$$\sum_{u \in U} u = n - m, \quad u \in \{0, 1\} \quad (6.2)$$

である. 左辺の各 u はそのセルが地雷なら 1, 安全なら 0 を表す.

- R1: $n - m = 0$ なら, U の全セルを安全とする.
- R2: $n - m = |U|$ なら, U の全セルを地雷とする.
- R3: $0 < n - m < |U|$ なら, 局所情報だけでは確定しない (制約をキャッシュする).
- 矛盾: $n < m$ または $n - m > |U|$ なら, 観測列または回路状態が不正である.

この処理を実装する `solver_neighbor_scan` は, `direction` (4 bit) を 0~7 へ進め, ST_ADDR → ST_READ の 2 状態で近傍 RAM を読む. `unknown_x[0:7]` と `unknown_y[0:7]` には未知近傍の座標を保持し, `unknown_mask[direction]` には該当方向を保存する. `known_mines` と `unknown_count` を 8 近傍ぶん集計した後, ST_CLASSIFY で R1/R2 を判定する. 確定セルは `result_valid`, `result_index`, `result_state` として 1 件ずつ出力され, 親 FSM が安全 FIFO および変更 FIFO へ同時に投入できたとき `result_ready` を返す.

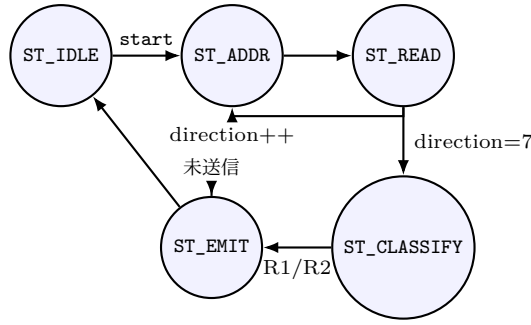


図 6.4: solver_neighbor_scan の FSM

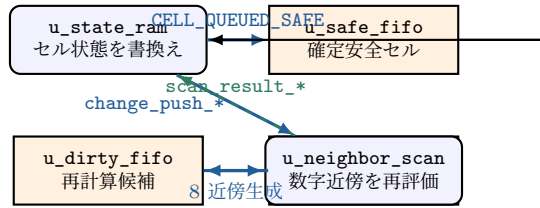


図 6.5: 変更伝搬と安全セル選択のキュー配線

例えば数字 3 の周囲に未知セルが 5 個あり，既知地雷が 1 個なら残りは 2 個である．このとき $0 < 2 < 5$ なので R3 となり，未知 5 セルのビットマスクと remaining=2 を solver_constraint_cache へ保存する．数字 3，既知地雷 3 個，未知セル 2 個なら remaining=0 で R1 となり，2 セルを CELL_QUEUED_SAFE へ書く．

6.4 安全・変更・dirty キューのハードウェア

推論結果を配列の再走査で処理すると，毎回 361 セルを調べる必要がある．そこで本設計は 3 本の固定容量 FIFO を使用する．安全セル用の solver_safe_fifo，状態が変化したセル用の solver_dirty_fifo（インスタンス名 u_change_fifo），数字セル再計算用の同 FIFO（u_dirty_fifo）である．

各 FIFO は queue_mem[0:360]，9 bit の head / tail，9 bit の count，361 bit の bitmap を持つ．安全 FIFO の queued_bitmap，dirty FIFO の pending_bitmap が既登録セルを検出し，push_duplicate を返す．同じセルを重複投入するのはエラーではなく，既に同じ仕事が待っているため無視できる状態である．一方，満杯で重複でもない投入は overflow_error をラッチする．

FIFO は push と pop が同一クロックに成立する 2'b11 も処理する．その場合は queue_mem[tail] を書き込み，tail と head を同時に進めるため，容量を一時的に増減させない．ポインタは 361 の次に 0 へ戻る円環構造である．

状態が変化したセル (x, y) を u_change_fifo から取り出すと，親 FSM の S_CHANGE_WALK で方向 0~8 を changed_direction に保持し，8 近傍へ変換する．範囲内の各セルを u_dirty_fifo へ入れる．S_DIRTY_CHECK で取り出したセルが CELL_OPEN_SAFE なら，保存していた ram_read_clue を scan_clue へ移し，scan_start を 1 クロックだけ発生させる．これが変更箇所から制約を増分更新する dirty 機構である．

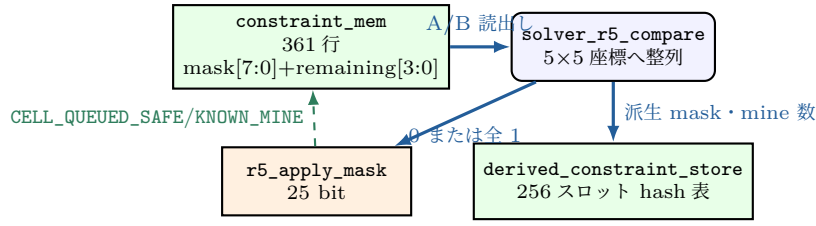


図 6.6: R5 のキャッシュ, 比較器, 派生制約ストア

6.5 グローバル推論 R4

局所数字を使わなくても, 盤面全体で残り地雷数が 0 または未知セル数と等しい場合には結論が出る. トップ FSM の S_R4_CHECK で, まず $\text{remaining_mines} > \text{unknown_count}$ を矛盾として検出する. 次に次の二つを判定する.

$$\text{remaining_mines} = 0 \Rightarrow \text{R4_ALL_SAFE}, \quad (6.3)$$

$$\text{remaining_mines} = \text{unknown_count} \Rightarrow \text{R4_ALL_MINE}. \quad (6.4)$$

S_R4_SCAN_ADDR $\rightarrow$ S_R4_SCAN_WAIT $\rightarrow$ S_R4_SCAN_APPLY で全セルを順番に RAM 読出しする. 未知セルだけを安全 FIFO / 既知地雷へ更新するため, 開示済みセルはスキップする. 全安全の場合は安全 FIFO が空になるまで S_POP_SAFE へ戻り, 全地雷の場合は S_FINISH へ進む.

6.6 制約キャッシュと R5 部分集合差分

R3 で得た局所制約を次のように表す. 数字セル c に対応する未知セル集合を A , 残り地雷数を a とすると, $A = a$ とは「集合 A の地雷数が a 」という意味である. 二つの制約 $A = a$, $B = b$ に対し, $A \subseteq B$ なら

$$B \setminus A = b - a. \quad (6.5)$$

を得る. 差集合の地雷数が 0 なら全セル安全, 差集合の要素数と同じなら全セル地雷, それ以外なら新しい派生制約となる. これを本レポートでは R5 と呼ぶ.

`solver_constraint_cache` は 361 行 $\times$ 12 bit (8 bit 未知マスク + 4 bit remaining) の同期 RAM である. `read_a_*` と `read_b_*` の 2 読出しポートにより, R5 比較器へ二つの制約を供給する. キャッシュ RAM 自体を毎回全消去せず, 親の 361 bit `constraint_valid` を新盤面開始時に 0 クリアする. S_SCAN_WAIT で近傍スキャンが完了すると, `scan_constraint_count` $\neq$ 0 の場合だけ `write_valid` を出し, その数字セルの制約を上書きする.

6.6.1 5 $\times$ 5 共通座標への変換

3 $\times$ 3 近傍マスクは数字セルごとの相対座標なので, そのままでは二つのマスクを AND できない. `solver_r5_compare` は二つの中心座標の小さい側から 1 を引いた点を 5 $\times$ 5 窓の `base_x, base_y` とし, 8 方向を同じ絶対座標へ埋め込んで `mask_a, mask_b` (各 25 bit) を作る.

例えば

$$A = \{p, q\}, \quad a = 1, \quad B = \{p, q, r, s\}, \quad b = 2 \quad (6.6)$$

なら、 $A \subseteq B$ をビット演算 $(\text{mask_a} \ \& \ \text{mask_b}) == 0$ で検出し、差分 $\{r, s\}$ と $\text{remaining}=1$ を得る。差分を 25 bit の $\text{r5_compare_difference_mask}$ に出力し、base 座標とともに保存または即時適用する。

比較器は同一マスクで remaining が異なる場合、差分の remaining が 0～要素数に収まらない場合を contradiction とする。非自明な差分は derived_valid として S_R5_STORE_WAIT へ渡す。

6.6.2 派生制約ストア

$\text{solver_derived_constraint_store}$ は 52 bit のエントリを 256 スロット持つ。内訳は 25 bit mask, 7 bit signed base-x, 7 bit signed base-y, 5 bit mines, 8 bit board-generation である。挿入時は ST_NORMALIZE_Y , ST_NORMALIZE_X で mask の左上の占有セルまで行列をシフトし、異なる 5×5 窓から同じ集合が得られたとき同じ表現になるようにする。

ハッシュは mask の各バイトの XOR と base 座標の XOR で 8 bit を生成し、衝突時は probe_index を 1 ずつ進める。 probe_count が $\text{MAX_PROBES}=32$ に達すれば overflow を立てる。既存エントリの mask, 座標, generation が一致し mine 数も一致すれば duplicate , mine 数だけ異なれば contradiction となる。

6.6.3 R5 の親 FSM 配線

S_R5_INIT は r5_center_x/y と $\text{r5_candidate_offset}$ を 0 にする。 S_R5_CANDIDATE では offset 0～24 を走査し、 constraint_valid が立つ二つのセルだけを比較する。適用可能な結論なら r5_apply_mask , r5_apply_is_mine , r5_apply_base_x/y ヘラッチする。 S_R5_APPLY_ADDR と S_R5_APPLY_WAIT で mask の最下位 1 ビットを選ぶ組合せ回路を待ち、 S_R5_APPLY でそのセルを 1 件ずつ RAM へ反映する。適用ごとに $\text{r5_apply_mask} \leftarrow \text{r5_apply_mask} \ \& \ \text{r5_apply_bit_mask}$ としてビットを消す。

安全判定なら安全 FIFO と変更 FIFO の両方が受け入れ可能になるまで r5_queues_ready を待つ。地雷判定なら安全 FIFO だけは不要だが、変更 FIFO は必須である。この配線により、状態 RAM の更新と再計算依頼が片方だけ成功する不整合を防ぐ。

6.7 ハードウェアとしての不変条件とエラー処理

ソルバは単に推論を行うだけでなく、入力ストリームが内部状態と矛盾していないかを検査する。代表的な不変条件を表 6.3 にまとめる。

エラー状態 S_ERROR では protocol_error を保持し、 $\text{busy}=0, \text{done}=1$ として終了する。例えば reveal_valid が 1 エントリの保留中にさらに到着した場合、 S_WAIT_DRAIN で検出してエラーとする。この設計は、異常入力を推論の一部として扱ってしまうより、原因を外部へ明示できる点で実験時のデバッグに有効である。

表 6.3: ソルバの主な不変条件

検査	実装上の条件
開示座標 状態遷移	<code>pending_reveal_x/y</code> が幅・高さ以内, <code>index</code> が 361 未満 未知セルからの安全／地雷反映, または既に予約された同種状態への反映 だけを許す
地雷数 制約	<code>remaining_mines <= unknown_count</code> , 0 未満になる遷移を禁止 数字より既知地雷が多い, または残り地雷が未知数を超える場合を矛盾と する
FIFO コンポーネント適用	重複は <code>bitmap</code> で無視し, 非重複の満杯投入は <code>overflow</code> として停止 <code>CELL_UNKNOWN</code> への新規結論, または同じ結論の再適用のみ許可

第 7 章

制約コンポーネントの構築と全列举回路

前章の R1-R5 は、局所的に安全／地雷を確定できるときは非常に強い。しかし、例えば「二つのセルのうち一つだけが地雷」という制約が複数個つながると、どのセルも単独では確定できない場合がある。本設計では、そのような未知セルを小さい連結成分に分け、成分内のすべての 0/1 割当をハードウェアで列举する。ただし回路資源を固定するため、成分の変数は最大 12 個、制約は最大 16 個（提出トップの候補収集時は MAX_COLLECTOR_CONSTRAINTS=8）に制限する。12 変数なら全割当は最大 $2^{12} = 4096$ 通りであり、深さ優先探索（DFS）を 1 クロックずつ進めれば固定回路で実現できる。

7.1 成分処理の全体配線

成分処理は、(1) 制約の座標変換、(2) 変数番号付けと制約格納、(3) 全列举、(4) 確定セルまたは候補の出力、という 4 ブロックに分かれる。親 FSM 内の `component_collector_mode` が通常の R5 ペア処理と、確率候補収集処理を切り替える。

上位の `solver_small_component_pipeline` はこれらを接続する。外部に対しては、`constraint_valid/ready`, `solve_start/ready`, `result_valid/ready` の 3 種類の `ready/valid` 境界を持ち、制約の発見と盤面状態への書込みは親の `minesweeper_solver_deterministic` が担当する。

7.2 局所マスクからセル番号への変換

`solver_component_adapter` は 5 状態（ST_IDLE, ST_BEGIN, ST_SCAN, ST_EMIT, ST_END）を持つ。入力が通常の数字制約なら `local_mask[7:0]` を 8 近傍として扱い、R5 派生制約な

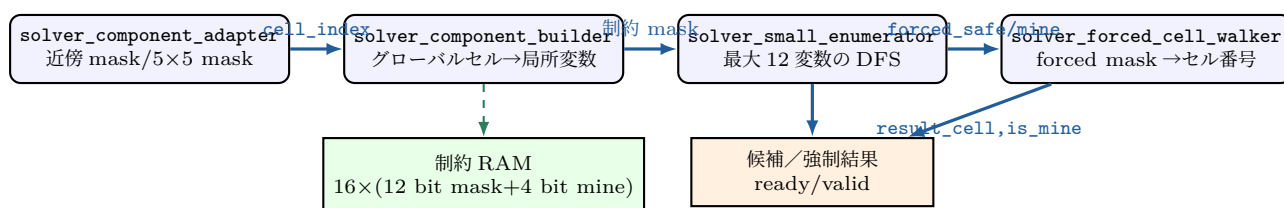


図 7.1: 小規模制約コンポーネントの処理パイプライン

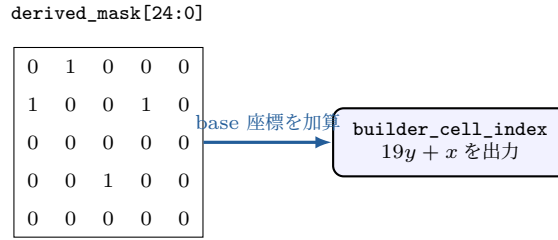


図 7.2: 派生制約の 5×5 マスクを疎なセル列へ変換するイメージ

表 7.1: solver_component_builder のデータ構造

配列／レジスタ	幅・深さ	役割
variable_cells	9 bit×12	局所変数番号からグローバルセル番号への逆引き表
constraint_masks	12 bit×16	各制約に現れる局所変数の集合
constraint_mines	5 bit×16	その集合に含まれる地雷数
current_mask	12 bit	1 件の入力制約を組立中の作業レジスタ
existing_variable_count	4 bit	制約開始前の変数数（切断時のロールバック用）
building,sealed	1 bit	制約入力中／入力完了後を示す制御レジスタ

ら derived_mask[24:0] を 5×5 窓として扱う．開始時に active_derived, active_width, active_height, active_center_x/y, active_base_x/y, active_mask, active_mines へ入力をラッチする．

通常制約の position 0～7 は次の相対座標へ変換される．

position	0	1	2	3	4	5	6	7
($\Delta x, \Delta y$)	(-1,-1)	(0,-1)	(1,-1)	(-1,0)	(1,0)	(-1,1)	(0,1)	(1,1)

派生制約では position[4:0] を ($position \bmod 5, \lfloor position/5 \rfloor$) として active_base_x/y に足す．cell_x_calc または cell_y_calc が有効盤面外なら coordinate_error を立てて中断する．選択されていない mask ビットは ST_SCAN で飛ばし，選択ビットだけを builder_cell_index へラッチして ST_EMIT で 1 件送る．このため，境界外のゼロ埋めを成分へ誤って混ぜない．

7.3 コンポーネント・ビルダの内部レジスタ

solver_component_builder は，ストリームで到着する複数の制約を，列挙器が扱える局所表現へ変換する．主な格納配列は次の通りである．

新しい制約の begin で current_mask=0, current_mines=begin_mines, existing_variable_count=variable_count とする．cell_valid && cell_ready ごとに，入力セルが variable_cells 内にあるかを最大 12 要素の組合せ比較で調べる．既存ならその変数ビットを立て，新規なら配列末尾へ格納し，variable_count を増やす．既存変数が 1 つでも含まれれば current_overlap を立てる．

end 時の判定は回路設計上重要である．

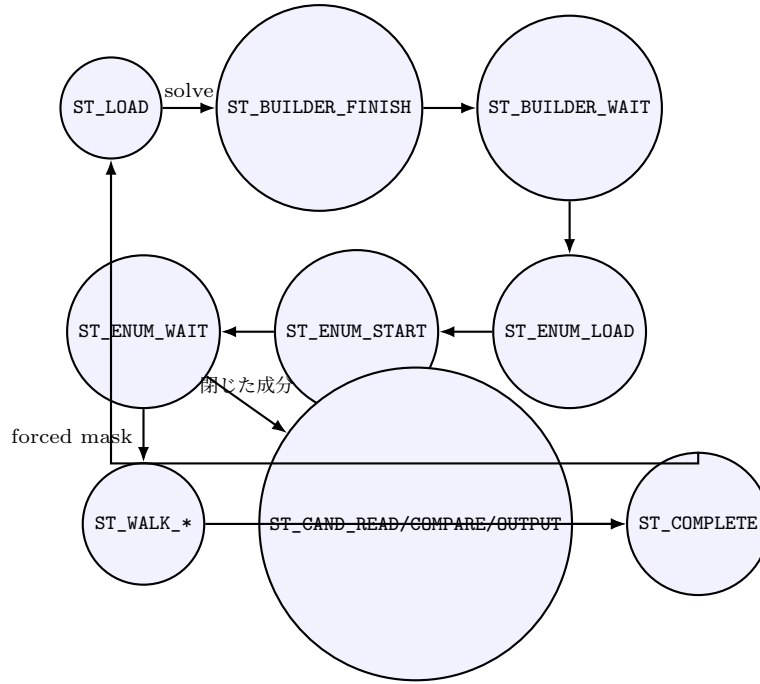


図 7.3: 小成分パイプラインの状態遷移

1. `current_mask==0` または `current_mines > count_ones(current_mask)` なら `malformed`.
2. 既存制約と重なりがなく, かつ既に成分制約が存在するなら `disconnected(SKIP_DISCONNECTED=1)` 時はこの制約だけ捨てる).
3. 同じ `mask` が既にあるのに `mine` 数が違えば `contradiction`.
4. 同一 `mask` がなければ `constraint_masks[constraint_count]` へ登録する.

成分を確定する `finish` で `sealed=1, component_valid=(variable_count!=0 ^constraint_count!=0)` となる. 12 変数を超えたときは, 通常モードなら `overflow` で停止し, 収集モード (`SKIP_DISCONNECTED=1`) なら `current_overflow` を立ててその制約を不完全として戻す. これにより, 大きなフロンティアが現れても回路全体が予期せず拡大しない.

7.4 小成分パイプラインの FSM

`solver_small_component_pipeline` の状態を図 7.3 に示す.

`ST_ENUM_LOAD` では `load_constraint_index` を 0 から `component_constraint_count-1` まで進め, ビルダの `query_constraint_mask/mines` を列挙器の `constraint_write_*` へ接続する. `ST_ENUM_WAIT` で列挙完了を待ち, `enum_forced_safe` または `enum_forced_mine` が非ゼロなら `walker` を起動する. どちらもゼロで `closed_local=1` なら, この成分は論理的には矛盾しないがセルを確定できないため, 候補選択用の `ST_CAND_*` へ進む.

7.5 DFS 列挙器のアルゴリズム

`solver_small_enumerator` は最大 12 変数の制約を正確に列挙する。各制約は 12 bit mask と 4 bit の地雷数であり、割当 `assignment[i]=1` を「局所変数 i が地雷」と定義する。有効割当 z は各制約 k について

$$\text{popcount}(z \& \text{mask}_k) = \text{mines}_k \quad (7.1)$$

を満たす必要がある。

7.5.1 内部 RAM と集計値

列挙器の主要な配列は次の通りである。

- `constraint_masks[0:15]`, `constraint_mines[0:15]` : 入力制約の RAM.
- `ways_by_mines[0:12]` : 地雷数ごとの有効割当数.
- `mine_ways[variable*(MAX_VARIABLES+1)+mine_count]` : 各変数が地雷で、かつ総地雷数が所定値の割当数.
- `mine_total[0:11]` : 各変数が地雷となった有効割当の総数.

`total_ways` は全有効割当数であり, `mine_total[i]=0` なら変数 i を安全, `mine_total[i]=total_ways` なら地雷と確定できる。

7.5.2 状態遷移と枝刈り

`assignment_depth` は次に決定する変数位置を示し, `mine_branch_pending` はまだ 1 側を試していない変数の bitmap である。ST_EXPAND で `assignment[depth]=0` を設定し, `mine_branch_pending[depth]=1` として深さを 1 増やす。深さが変数数に達すると ST_RECORD へ入り, `assignment` の popcount を `assignment_mines` へ求める。

ST_CHECK_NODE の枝刈りは二つの条件を持つ。部分割当の既知地雷数を p , 未割当で制約に現れる変数数を q , 制約の要求地雷数を r とすると,

$$p > r \quad \text{または} \quad p + q < r \quad (7.2)$$

なら, 残りの変数をどう選んでも成立しないため ST_BACKTRACK へ進む。この上限/下限検査を行うことで, 単純な 4096 通りの全探索より早く終了する。ただし, 回路のレイテンシは入力データに依存する。

7.5.3 列挙結果からの強制判定

ST_FINISH で各変数 i について

$$\text{mine_total}[i] = 0 \Rightarrow i \text{ は強制安全}, \quad (7.3)$$

$$\text{mine_total}[i] = \text{total_ways} \Rightarrow i \text{ は強制地雷} \quad (7.4)$$

表 7.2: solver_small_enumerator の DFS 状態

状態	主要レジスタ	処理
ST_IDLE	入力	start で変数数, 制約数, 最大地雷数をラッチ
ST_CLEAR	clear_mine_count	集計 RAM を 0 クリア
ST_VALIDATE	constraint_cursor	mask が有効範囲か, mine 数が多いすぎないか検査
ST_CHECK_NODE	assignment_depth	部分割当が制約を満たし得るか判定
ST_CHECK_CONSTRAINT	pending_constraint_mask	残り制約を最大 2 件ずつ枝刈り検査
ST_EXPAND	assignment, mine_branch_pending	現在変数を安全 (0) 側へ展開
ST_RECORD	集計 RAM	完全割当を地雷数別に記録
ST_BACKTRACK	backtrack_depth	保留中の地雷 (1) 枝へ戻る
ST_FINISH	forced mask	total_ways から強制セルを生成

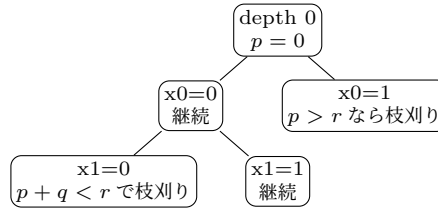


図 7.4: DFS における上限・下限枝刈りの概念例

と判定し, 12 bit の forced_safe_mask, forced_mine_mask を出力する. 有効割当が 0 通りなら contradiction を立てる. 制約数が 2 未満の場合は use_dfs=0 として互換用の単純割当列挙状態を使い, 制約を 1 件ずつ完全一致で検査する.

7.6 強制セル walker と親ソルバへの書込み

列挙器の mask は局所変数番号であり, 盤面のセル番号ではない. solver_forced_cell_walker は query_variable を 0 から active_variable_count-1 まで進め, ビルダの query_cell を読み出す. active_safe_mask または active_mine_mask が 1 なら ST_EMIT で result_cell と result_is_mine を ready/valid 出力する. 同じ変数が両方の mask に立っていれば矛盾である.

パイプライン側が結果を受けると, component_apply_index と component_apply_is_mine へラッチし, S_COMP_APPLY_ADDR → S_COMP_APPLY_WAIT → S_COMP_APPLY で状態 RAM を同期読出しする. 未知セルなら新たに CELL_QUEUED_SAFE または CELL_KNOWN_MINE を書き, 安全の場合は u_safe_fifo へも投入する. 既に同じ結論が書かれている場合は component_apply_compatible で許可される.

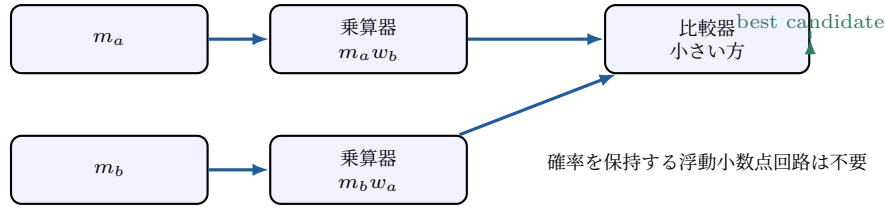


図 7.5: 確率候補を交差積で比較するデータパス

7.7 成分の収集と重複抑制

論理的に確定する成分を処理した後、親 FSM は S_COLLECT_INIT へ進み、collector_seed_index=0 から有効制約をセル番号順に調べる。collector_scan_index 周辺の 2 スロットを組合せ優先エンコード (collector_scan_lane=0,1) で調べ、空の 361 bit 配列を毎回フルスキャンしない。

シード制約を S_COLLECT_LOAD へ入力し、それと重なる制約をパスごとに追加する。collector_pass_added が 1 ならもう一度 scan index を 0 へ戻し、追加がなかった時点で成分を閉じる。collector_source_0~3 と collector_source_count には成分を構成した最大 4 個の制約番号を保存する。閉じた成分のシグネチャ collector_signature_0 は、次のシードが同じ成分を再処理しないための 4 スロット比較に使う。

成分に制約を追加するたび、ビルダは以前の変数との重なりを検査する。新規制約に既存変数がない場合は切断成分なので、収集モードではその制約を捨てて constraint_incomplete を返す。この処理は、列挙器の最大変数数を超える巨大成分を無理に一つの DFS へ送らないための安全策である。

7.8 確率候補の生成と比較回路

強制 mask が空で、成分が閉じている場合、列挙器は変数ごとの地雷出現数を候補として出力する。pipeline の ST_CAND_READ は candidate_variable を使って builder_query_cell を読み、enum_query_mine_total を candidate_current_mines ヘラッチする。ST_CAND_COMPARE では、地雷出現数が小さい候補を保存し、同数ならグローバルセル番号が小さい方を選ぶ。親ソルバへは candidate_cell, candidate_mine_ways, candidate_total_ways を出力する。

親 FSM は全成分から得た候補 (m_i, w_i) (セル i が地雷である割当数 m_i , 成分の総有効割当数 w_i) を比較する。確率 m_i/w_i の比較で除算器を使わず、次の交差積を用いる。

$$\frac{m_a}{w_a} < \frac{m_b}{w_b} \iff m_a w_b < m_b w_a. \quad (7.5)$$

RTL では candidate_compare_left = component_candidate_mines * best_candidate_ways, candidate_compare_right = best_candidate_mines * component_candidate_ways という乗算器 2 個を組み合わせた回路として構成し、等しい場合は component_candidate_cell < best_candidate_cell でタイブレークする。

この比較は「最も安全そうなセル」を選ぶためのものであり、そのセルが必ず安全だと証明するものではない。したがって、候補を選んだ後も通常の S_ISSUE → 開示 → 推論ループへ戻る。

表 7.3: 数学的対象と RTL 要素の対応

数学・アルゴリズム	ハードウェア表現	主な信号／レジスタ
未知セル集合	8 bit または 25 bit mask	unknown_mask, difference_mask
制約の残り地雷数	4～5 bit 整数	remaining, constraint_mines
局所制約の集合	361 行同期 RAM	constraint_mem, constraint_valid
派生制約集合	256 スロット線形探索 hash 表	slot_valid, probe_index
連結成分	最大 12 セルの番号表	variable_cells
割当の集合	DFS 状態レジスタ	assignment, assignment_depth
未試行の地雷枝	bitmap	mine_branch_pending
確定セル集合	2 本の mask	forced_safe_mask, forced_mine_mask
確率比較	整数交差積比較	candidate_compare_left/right

7.9 回路資源とアルゴリズムの対応

以上のように、集合演算は固定幅ビットマスク、集合の要素数は popcount 相当の組合せ加算、探索木は assignment と assignment_depth のレジスタ、待ち仕事は bitmap 付き FIFO として実装されている。動的メモリや再帰関数は用いず、最大サイズをパラメータで切ることによって RTL 合成可能な回路へ変換している。

第 8 章

行き詰まり時の選択戦略と確率フィードバック

論理推論は、盤面の情報が少ない序盤や、複数の制約が独立している場面では次の安全セルを一意に決められない。本設計では、論理推論を無限に待つのではなく、決められた順序で「角の開示」、「小成分の候補確率」、「辺中央の救済選択」を試し、それでも選べなければ *stalled* として正常終了する。*stalled* は RTL のクラッシュや通信エラーではなく、「現在観測した制約から許可された選択を使い切った」という結果である。

提出トップ `rtl/top/minesweeper_jtag_submission_local_top.v` は次のパラメータでこの戦略を有効にする。

8.1 行き詰まりの定義と制御フロー

トップ FSM の `S_R4_CHECK` で、安全 FIFO、変更 FIFO、dirty FIFO が空になるまで処理を続ける。全体地雷数から安全／地雷が一意に決まらなければ `R5` の比較を行い、それでも変更がなければ `S_OPEN_NEXT` へ入る。`S_OPEN_NEXT` では角戦略が残っていれば次の角を設定し、それも終われば `S_COLLECT_INIT` へ進む。成分収集が終わって候補が得られなければ、確率フィードバックまたは辺中央救済を調べる。いずれも許可されなければ `stalled<=1'b1` として `S_FINISH` へ進む。

表 8.1: 提出トップの行き詰まり対策パラメータ

パラメータ	値	意味
<code>ENABLE_CORNER_ON_STALL</code>	1	初手以外の角を試す
<code>MAX_PROBABILITY_GUESSES</code>	15	1 盤面での候補選択上限
<code>BASE_PROBABILITY_GUESSES</code>	8	通常の候補選択回数
<code>ENABLE_SATURATING_HALF_SAFE_FEEDBACK</code>	1	安全セル半数未満の間だけ追加候補を許す
<code>ENABLE_LOW_SCORE_EDGE_RESCUE</code>	1	安全開示 16 未満で辺中央を試す
<code>LOW_SCORE_RESCUE_SAFE_THRESHOLD</code>	16	救済選択を許す安全セル数の上限

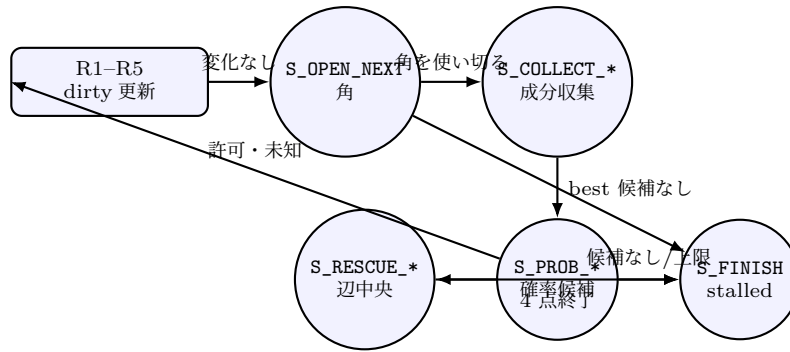


図 8.1: 論理推論が止まった後の選択制御

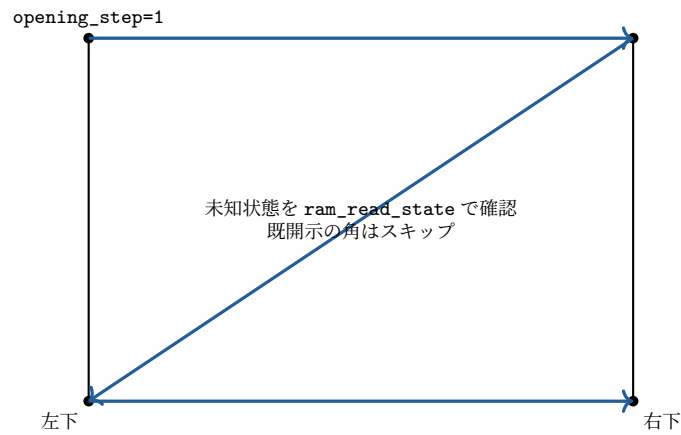


図 8.2: 角選択の順序（初手を含む）

8.2 角選択

盤面の角は、最初の開示による 0 セル連鎖を起こしやすく、座標の境界処理も単純である。初手は $\text{opening_step}=1$ で $(0, 0)$ を選ぶ。最初の推論が止まったとき S_OPEN_NEXT は次の順に active_x/y を設定する。

1. $\text{opening_step}=1 : (\text{width} - 1, 0)$ (右上).
2. $\text{opening_step}=2 : (0, \text{height} - 1)$ (左下).
3. $\text{opening_step}=3 : (\text{width} - 1, \text{height} - 1)$ (右下).

各候補は直ちに選択せず、 $S_OPEN_READ_ADDR \rightarrow S_OPEN_READ_WAIT \rightarrow S_OPEN_CHECK$ で u_state_ram の状態を読む。CELL_UNKNOWN なら S_ISSUE へ進み、既に開示済みまたは予約済みなら次の角へ進む。この 3 状態は同期 RAM の 1 サイクル遅延を吸収するために必要である。

角選択で新しい開示が得られると通常の S_WAIT へ戻り、その開示から R1-R5 を再開する。4 角を調べ終えてなお結論がなければ、角選択だけで成功したとはみなさず、成分収集へ進む。

表 8.2: 確率候補に関するレジスタ

レジスタ	幅	役割
probability_guess_count	4 bit	盤面内の確率選択回数 (0~15)
best_candidate_cell	9 bit	現時点で最も低い地雷比のセル
best_candidate_mines	13 bit	上記セルの M_i
best_candidate_ways	13 bit	上記成分の W_i
component_candidate_*	32 bit 級	現在入力された成分候補
active_x/y	5 bit×2	実際に ISSUE する座標

8.3 成分候補の選択

8.3.1 候補の意味

成分列挙で有効割当数を W ，セル i が地雷となる割当数を M_i とすると，セル i の局所地雷確率は

$$P_i = \frac{M_i}{W}. \quad (8.1)$$

solver_small_component_pipeline はこの分数を浮動小数点化せず, candidate_mine_ways= M_i と candidate_total_ways= W のまま親へ渡す. 親 FSM の best_candidate_* が成分ごとの最小候補を保持し, 交差積

$$M_i W_{best} < M_{best} W_i \quad (8.2)$$

で比較する. 同率ならセル番号が小さい方を採用する. best_candidate_valid が 0 の間は最初の閉じた成分候補を無条件に保存する.

8.3.2 候補探索のレジスタと状態

候補収集後, S_COLLECT_SEED で collector_seed_index $\geq$ MAX_CELLS になると候補選択へ進む. 候補があれば active_x $\leq$ best_candidate_cell % 19, active_y $\leq$ best_candidate_cell / 19 とし, S_PROB_READ_ADDR, S_PROB_READ_WAIT, S_PROB_CHECK で未知状態を確認する. CELL_UNKNOWN なら probability_guess_count を増やして S_ISSUE へ進む. 成分処理中に別の制約がセルを確定させていた場合は, 候補を再計算せず安全に stall 終了する.

8.4 候補回数の上限と半数条件

提出版では通常 8 回の確率候補を許可する. 条件式は

$$\text{probability_guess_count} < 15 \quad \wedge \quad \text{probability_guess_count} < 8 \quad (8.3)$$

であり, カウンタ 0~7 の計 8 回である. ただし ENABLE_SATURATING_HALF_SAFE_FEEDBACK=1 なので, 8 回に到達した後も安全セルの半数未満なら追加候補が許可される.

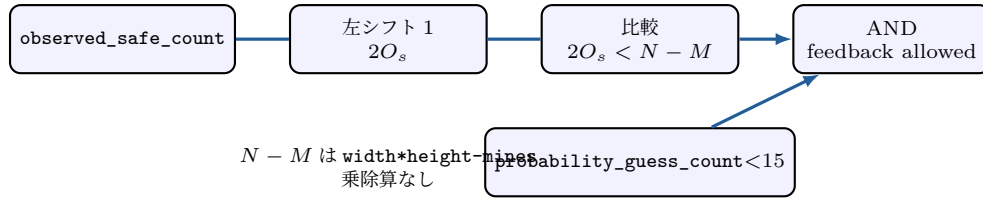


図 8.3: 安全セル半数条件の組合せデータパス

全セル数を $N = width \times height$, 総地雷数を M , 開示済み安全セル数を O_s とすると, 条件は

$$2O_s < N - M. \quad (8.4)$$

RTL では `half_safe_opened_twice = observed_safe_count << 1`, `half_safe_total = adaptive_board_cells - mines` として比較する. 除算器を使わないため, 候補継続の判断が一組の加算／比較回路になる. `probability_guess_count` は 4 bit なので 15 で飽和し, 8 回以後に継続しても最大 15 回で止まる.

なお, `ENABLE_ADAPTIVE_FEEDBACK=0` のため, 提出版では `ADAPTIVE_FEEDBACK_SAFE_THRESHOLD=32` と確率リスク交差積による経路は使用しない. RTL には将来の実験用として残っており, パラメータを 1 にすれば, 8 回後かつ安全開示 32 未満で, 候補の期待リスクが全体条件より小さい場合に追加選択できる.

8.5 辺中央救済選択

成分候補がない場合, 安全開示数が 16 未満であれば, `S_RESCUE_NEXT` が 4 点を試す. `rescue_step` に応じた座標は次の通りである.

0 : ($\lfloor width/2 \rfloor, 0$)	上辺中央,
1 : ($\lfloor width/2 \rfloor, height - 1$)	下辺中央,
2 : ($0, \lfloor height/2 \rfloor$)	左辺中央,
3 : ($width - 1, \lfloor height/2 \rfloor$)	右辺中央.

`rescue_step` は 3 bit で保持し, 4 点を超えない. 各点は角と同じく `S_RESCUE_READ_ADDR` → `S_RESCUE_READ_WAIT` → `S_RESCUE_CHECK` で状態 RAM を確認する. 未知なら選択し, 既開示なら次へ進む. 4 点すべてが使えなければ `stalled` を立てる.

辺中央は論理的な安全保証を持つ点ではなく, 序盤の観測数を増やすための経験的ヒューリスティックである. したがって, 本処理で地雷を選んでも `observed_mine_count` に記録され, スコア計算ではペナルティになるが, 回路エラーにはならない.

8.6 推測とスコアの関係

候補選択および救済選択は「安全セルを証明した」結果ではない. 選択して地雷が開示されても, `CELL_OPEN_MINE` への遷移は合法であり, `observed_mine_count` を 1 増やして次の推論へ戻る. 一方, すべての安全セルが開示された場合でも, 未確定 UNKNOWN が残ることがある.

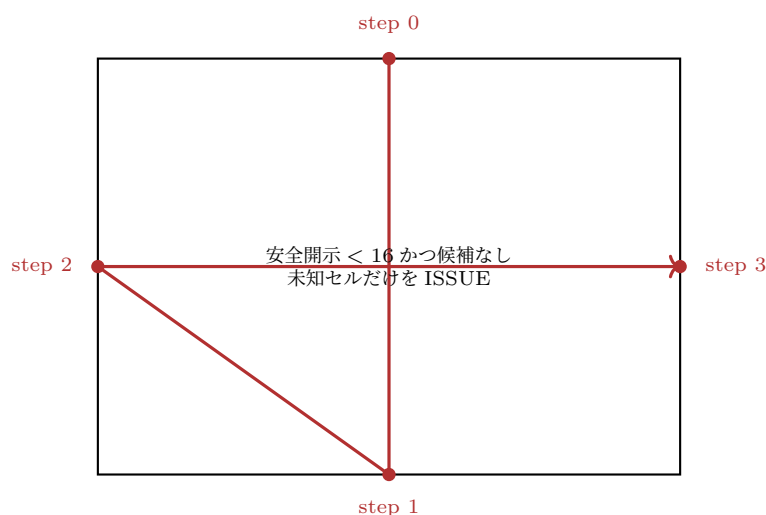


図 8.4: 辺中央救済の 4 候補

S_WAIT_DRAIN は安全数が $width \times height - mines$ に達したことを検出すると、R4 全地雷を適用してから終了する。

この区別により、次の三つの結果を分けて評価できる。

1. 全安全セル開示：完全解答。
2. 地雷を開示したが、その後に安全セルを開示：推測を含むが処理継続。
3. 制約、候補、救済を使い切った：stalled=1 で正常終了。

protocol_error=1 となる不正入力（座標外、開示の二重到着、キュー overflow など）とは異なり、stalled は盤面の難しさに由来するアルゴリズム上の出力である。実機結果 CSV ではこの二つを別列として集計すべきである。

8.7 戦略のハードウェア上のトレードオフ

確率選択の上限を増やすと完全解答率が向上する可能性がある一方、地雷を開く機会と処理クロックが増える。本設計の上限は 4 bit カウンタで実装できる 15 回であり、候補の確率比較は除算器を使わず交差積の乗算器で行う。半数条件はシフトと比較だけである。角／辺選択は確率演算を増やさず、固定個数の状態遷移と状態 RAM 読出しだけで構成できる。

また、候補を選ぶ前に best_candidate_valid をクリアし、成分ごとの候補を逐次比較するため、候補全件を RAM へ保存する必要がない。一方、成分が 12 変数を超える場合は列挙回路を拡張せず収集から除外するため、そのような盤面では stalled になりやすい。これは「有限の論理回路で、最悪ケースのリソースを予測可能にする」という FPGA 設計上の選択である。

第 9 章

JTAG 通信とクロックドメイン交差

9.1 通信の役割

実機では USB-Blaster を介して PC から盤面を送り、処理結果を読み出す。FPGA 側のトップは minesweeper_jtag_submission_local_top であり、JTAG の 64 ビットデータレジスタを一つのメールボックスとして使用する。通信層は、(i) Virtual JTAG の TCK で 64 ビットをシフトする virtual_jtag_shift64, (ii) TCK とシステムクロック間でコマンドを受け渡す virtual_jtag_mailbox, (iii) コマンドを内部ストリームへ変換する jtag_protocol_engine, (iv) 盤面の設定・セル転送・開始を管理する board_stream_controller からなる。

9.2 64 ビットコマンド形式

コマンドの opcode は command_word[63:60] に置く。主要な opcode を表 9.1 に示す。BEGIN は盤面 ID, 幅, 高さ, 地雷数を指定し, DATA は最大 11 セル (data_remaining は 4 ビット) を 4 ビットセル値で詰める。COMMIT の下位 16 ビットは CRC-16-CCITT であり, エンジン は running_crc と比較してから開始する。

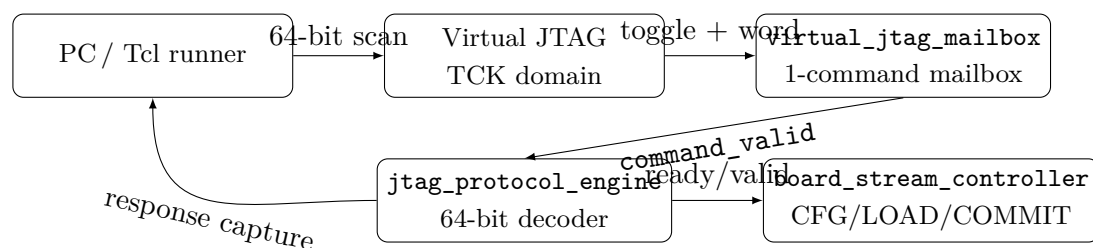


図 9.1: JTAG から盤面ストリームへの信号経路



図 9.2: 64 ビットワードの概念図 (フィールド幅は opcode ごとに再利用される)

表 9.1: JTAG opcode と応答

opcode	命令	応答・作用
0	NOP / STATUS	状態ビットを返す
1	BEGIN	begin_board_id,width,height,mines を発行
2	DATA	cell_ordinal とセル列を発行
3	COMMIT	CRC を検査して開始
4	STATUS	busy, 結果有効, エラーを返す
5	RESULT_A	盤面属性, 開示数, 選択数
6	RESULT_B	サイクル数と符号付きスコア
7	ACK	結果スナップショットを解放
8	VERSION	プロトコル version=2, BUILD_ID=0001
9	PING	入力下位 60 ビットを XOR して返す
A	RESULT_C	選択数, stalled

9.3 TCK 側のシフタ

virtual_jtag_shift64 は shift_reg[63:0], shift_count[6:0], scan_active を持つ。Capture-DR (virtual_state_cdr) で応答を shift_reg へ取り込み, Shift-DR (virtual_state_sdr) ごとに shift_reg <= {tdi,shift_reg[63:1]}を実行する。63 回目のシフトで received_word を確定し, received_toggle を反転する。したがって Update-DR を観測する必要がなく, Tck 側のスキャン終了時点で一意にコマンドを通知できる。途中で Capture を再実行した場合や 64 ビット未満で終了した場合は scan_error を立てる。

9.4 CDC メールボックス

JTAG の TCK はシステムの 20 MHz クロックと位相・周波数が同期していない。このため 64 ビットバスを直接サンプリングせず, データを TCK 側の tck_mailbox に保持してから 1 ビットのトグルを同期する。システムクロック側では 3 段レジスタ post_sync[2:0] へ post_toggle を通し, post_sync[2] != seen_toggle を新規コマンドとして検出する。バスはトグル検出時まで保持されているため, データの安定時間を確保できる。command_valid が command_ready と同時に成立したときだけ消費し, 1 コマンド処理中に次の投稿を受けない。この構造により, 異なるクロックドメイン間でパルスを直接渡すことを避けている。

9.5 プロトコルエンジンとバックプレッシャ

jtag_protocol_engine の実行状態は EX_IDLE, EX_BEGIN, EX_DATA, EX_COMMIT である。command_ready は exec_state==EX_IDLE のときだけ 1 となる。例えば DATA では cell_valid を保持し, cell_valid && cell_ready のクロックで CRC 更新, data_values の 4 ビット右シフト, cell_ordinal の加算を行う。下流の load_ready が 0 なら上流のセルは進まず, ready/valid

によるバックプレッシャが成立する.

`result_snapshot` は結果を `available` が 1 の間保持する. 未 ACK のまま次の結果を capture すると `overflow_error` を立てるため, ホストは `RESULT_A/B/C` を読み, ACK を発行してから次の `BEGIN` を送る.

第 10 章

評価値計算と表示回路

10.1 スコアの定義と固定小数点化

`minesweeper_metrics` は盤面完了信号 `board_done` を受けて、開いた安全セル数を `current_safe`, 開いた地雷数を `current_mines` ヘラッチする. 盤面の安全セル総数を $N_s = WH - N_m$, 地雷総数を N_m とすると, スコアは

$$S = \frac{O_s}{N_s} - \frac{O_m}{N_m} = \frac{O_s N_m - O_m N_s}{N_s N_m} \quad (10.1)$$

である. RTL では `safe_product=current_safe*mines_latched`, `mine_product=current_mines*safe_cells`, `rational_difference` を生成し, 分子全体を 10000 倍してから一度だけ除算する. この順序により, 各分数を先に丸める誤差を避ける. `current_score_scaled` と `total_score_scaled` は符号付き 32 ビットで, 例えば 9259 は 0.9259 を表す.

10.2 メトリクス FSM

`minesweeper_metrics` は `S_IDLE`, `S_SAFE_START`, `S_SAFE_WAIT`, `S_COMMIT` を持つ. `IDLE` で盤面情報と `board_cycles` を取り込み, `total_cycles` ヘ加算する. `SAFE_START` で `divider_numerator` と `divider_denominator` を設定し, `divider_start` を 1 クロックだけ発行する. `SAFE_WAIT` は `divider_done` を待ち, `COMMIT` で商の符号を戻して結果を確定する. `current_safe==safe_cells` なら `boards_fully_solved` を加算し, `result_valid` を 1 周期出力する.

10.3 逐次除算器

`unsigned_divider` は組合せ除算器を使わない 32 ビット `restoring divider` である. 内部レジスタは `dividend_work`, `divisor_work`, `quotient_work`, 33 ビットの `remainder_work`, 6 ビット

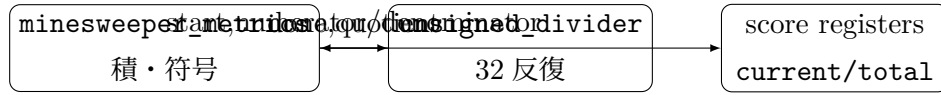


図 10.1: スコア計算のデータパス

の iteration である。毎クロック

$$R' = \{R[31:0], D[31]\}, \quad (10.2)$$

$$q = [R' \geq M], \quad (10.3)$$

$$R_{next} = q ? R' - M : R', \quad (10.4)$$

$$Q_{next} = \{Q[30:0], q\} \quad (10.5)$$

を実行し、32 回目に quotient と remainder_out を出力する。denominator==0 の場合は divide_by_zero を立て、商を 32'hffff_ffff として即時完了する。

10.4 7 セグメント表示

minesweeper_display_formatter は 64 桁の表示ビット列 segments[511:0] を生成する。状態は ST_CLEAR, ST_LOAD, ST_CONVERT, ST_WRITE, ST_COMMIT であり、13 フィールドを順番に処理する。binary_work と bcd_work を 32 回シフトし、5 以上の BCD 桁へ 3 を加える double-dabble を実装している。これにより大きな組合せ除算器を表示経路に置かずに済む。フィールドごとの値は処理数、完全解答数、盤面番号、選択数、安全開示数、地雷開示数、幅、高さ、地雷数、盤面サイクル数、累積サイクル数、現在スコア、累積スコアである。

mu500_7seg_latch_driver は ST_SETUP, ST_LATCH, ST_HOLD で動作する。64 本の共有データ線 data_lines を SEG_A~SEG_H へ分割し、SEG_SEL[8:0] の SEL0~SEL7 で 8 面の 7 セグメント列、SEL8 で 64 個の LED をラッチする。データを先に設定してから選択線を 1 にするため、ラッチ中の桁化けを抑制する。表示範囲を超えた場合は overflow を LED 経路へ伝える。

10.5 速度制御

solver_speed_control はクロックをゲートせず、run_enable を FSM の clock enable として使用する。設定 F では毎クロック実行し、0 では停止する。1~E では CLOCK_HZ/divider で実行間隔を作る。この方式はクロックのスキューや合成上のゲーティング問題を避け、表示や通信だけを継続できる。

第 11 章

シミュレーション，実機試験，ビルド 設定

11.1 RTL シミュレーション

シミュレーションのファイルリストは `sim/official.f` で管理し, `sim/run_official_file.ps1` が Icarus Verilog のコンパイルと実行を自動化する. スクリプトは `-g2012 -Wall -s tb_official_board_file` を指定し, `+BOARD_FILE`, `+RESULT_CSV`, `+START_BOARD`, `+MAX_BOARDS` をシミュレータへ渡す. 決定論ソルバを選ぶ場合は `-Ptb_official_board_file.DETERMINISTIC_SOLVER=1` を追加し, 確率試行回数, adaptive feedback, edge rescueなどをパラメータ化できる.

`tb_official_board_file.sv` は公式盤面のヘッダを読み, `begin_valid/ready`, `cell_valid/ready`, `commit_valid/ready` を順に駆動する. 結果を受けた後は `result_snapshot` の各フィールドを照合し, スコア分子・分母から再計算した値と RTL の `score_scaled` が一致しなければエラーとする. 出力 CSV には設定時間, ロード時間, ソルバ時間, スコア時間, 総サイクル, プロファイル別サイクル, コンポーネント統計, 列挙ノード数まで記録できる.

11.2 Virtual JTAG 実機試験

`sim/jtag_official_runner.tcl` は Quartus SignalTap スクリプト環境から USB-Blaster を操作する. 最初に Virtual JTAG の version を読み, プロトコル version 2 と `BUILD_ID=0001` を確認する. 各盤面について BEGIN, DATA 列, CRC 付き COMMIT を送り, STATUS をポーリングして `result_available` を待つ. RESULT_A/B/C を読み, ACK を送ってスナップショットを解放する. Tcl 側の CSV には `board_crc`, `jtag_retries`, `transport_error`, `protocol_version`, `build_id` も保存されるため, アルゴリズム結果と通信障害を区別できる.

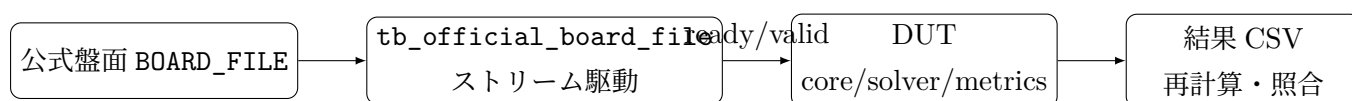


図 11.1: 公式ファイルシミュレーションの検証経路

11.3 Quartus プロジェクト

提出用 QSF はトップレベルを minesweeper_jtag_submission_local_top, 出力ディレクトリを output_submission_local_mu500, 最適化を AGGRESSIVE AREA, I/O 規格を 3.3-V LVTTL と指定する. SDC の create_clock -name CLK_20MHZ -period 50.000 [get_ports {CLK_20MHZ}] が 20 MHz 制約を与え, derive_clock_uncertainty で不確実性を導出する. 実機では生成した SOF を Quartus Programmer で書き込み, その後 Tcl runner を実行する.

11.4 検証項目

主なチェックは次の通りである.

1. 盤面ヘッダの幅・高さが 1~19, 地雷数が 0 より大きくセル数未満であること.
2. DATA の ordinal が 0 から連続し, セル値が 0~9 であること.
3. CRC 不一致, 結果上書き, 重複開示, プロトコル状態不整合をエラーにすること.
4. 結果スコアを整数有理式から再計算し, RTL 出力と一致させること.
5. シミュレーションと実機で同一盤面のフィールドが一致すること.

第 12 章

実機評価と考察

12.1 speed10 の結果

提出 FPGA を 20 MHz で動作させ、`speed10_hardware_1000.csv` の 1,000 盤を集計した。結果を表 12.1 に示す。

20 MHz を仮定すると平均処理時間は $81110.615 / (20 \times 10^6) = 4.0555 \text{ ms}$ である。ここで CSV の `cycles` は FPGA 内の盤面処理サイクルであり、PC からの JTAG 転送時間や Tcl の待ち時間を含む壁時計時間ではない。

12.2 スコア分布

各盤面のスコアは `score_scaled/10000` として保存される。図 12.1 はレポート生成時に CSV から作成するヒストグラムの配置例であり、1.0 付近に完全解答が集まり、`stalled` 盤面では安全セルを多く開けても地雷開示ペナルティにより低下する。

12.3 サイクル数の解釈

盤面ごとの処理量は盤面サイズ、開示連鎖、制約コンポーネントの大きさに依存する。特に全列挙は変数数を k とすると候補上限が 2^k であり、`solver_small_enumerator` の最大 12 変数では 4,096 通

表 12.1: speed10 実機 1,000 盤の集計

実行盤面数	1000
完全解答 (<code>opened_safe</code> =安全セル総数)	862
<code>solver_stalled=1</code>	138
地雷を 1 個以上開いた盤面	592
平均スコア	0.9038487
最低／最高スコア	0.0116 / 1.0000
総サイクル数	81,110,615
平均サイクル数	81,110.615
最大サイクル数	1,310,106

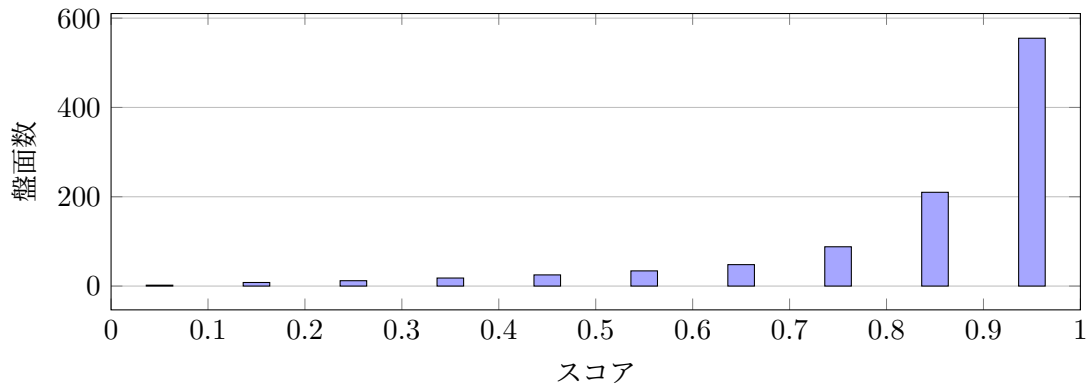


図 12.1: スコア分布の描画例（最終版では CSV から自動生成する）

りとなる．制約の多い盤面では `profile_builder`, `profile_enumeration`, `profile_candidate` のサイクルが増え，最大値 1,310,106 サイクルの要因になり得る．一方，安全セルの連鎖開示が大きい盤面では `minesweeper_game_core` の FIFO 走査が支配的になる．

12.4 stalled の意味

`stalled` は通信異常ではない．`result_stalled` は，局所制約，R4 全体制約，R5 差分制約，列挙結果，許可された確率選択を使っても次の選択を一意に決められないことを表す．したがって 138 盤では処理はプロトコル上正常終了している．一方，`transport_error` や `protocol_error` が 0 であることにより，アルゴリズム上の停止と実装上の通信エラーを分離できる．

12.5 考察

完全解答率 86.2% と平均スコア 90.38487% の差は，安全セルをすべて開けられなかった盤面，または地雷を開けた盤面が含まれるためである．R5 の部分集合差分と最大 12 変数の全列挙は局所的な曖昧さを減らす，互いに分離したコンポーネントを残し地雷数で完全結合するグローバル条件付けは現行提出設定で無効である．そのため，数学的には利用可能な情報が残っていても，回路の固定容量・処理時間上限により `stalled` となる場合がある．

第 13 章

回路実装上の工夫

13.1 固定サイズメモリ

最大 $19 \times 19 = 361$ セルに対して, セルアドレスは $19y + x$ とする. RTL の `board_stream_controller` は `load_index = (load_y << 4) + (load_y << 1) + load_y + load_x` で計算し, 乗算器を使わない. `board_mem[0:360]`, 開示ビットマップ, 未知・既知地雷状態を固定容量にすることで, 入力盤面によって回路規模が変わらない.

13.2 逐次化とハンドシェイク

全処理を一クロックで組合せ評価せず, FSM と `ready/valid` で分割した. ゲームコアの開示連鎖, neighbor scan, component builder, enumerator はいずれもアドレス, カウンタ, 待ち状態をレジスタに保持する. RAM 読み出しの 1 クロック遅延を前提にすることで, 推論結果を安定した信号として次段へ渡せる.

13.3 キューと bitmap

`solver_safe_fifo`, `solver_dirty_fifo`, ゲームコアの `queue_mem` は, 登録と取り出しを別のクロックで行う円環 FIFO である. `pending` や `scheduled` のビットマップは, 同じセルを何度もキューへ入れることを防ぐ. これは論理演算を少し追加する代わりに, キューのオーバーフローと再計算回数を抑える設計である.

13.4 全列挙の上限と枝刈り

`solver_component_builder` は最大 12 変数・16 制約の成分を構成し, `solver_small_enumerator` は深さ優先で 2 値割当を生成する. 部分割当の時点で制約の最小・最大可能地雷数が要求値に届かない場合, その枝を打ち切る. この上限は完全性を限定する一方, 指数処理が無制限に回路を占有することを防ぐ.

13.5 除算を避ける比較

確率候補の比較は分母を実際に割らず，交差積で行う．候補 a, b について $m_a/n_a < m_b/n_b$ を $m_a n_b < m_b n_a$ と比較すればよい．同様に安全セル半数判定は $2O_s < N_s$ とし，除算器を追加しない．除算が必要な表示・スコア確定時だけ `unsigned_divider` を共有する．

13.6 観測面と完全盤面の分離

`minesweeper_game_core` だけが地雷値を含む `board_mem` を持ち，ソルバへ渡るのは `reveal_x`, `reveal_y`, `reveal_is_mine`, `reveal_clue`, `cascade_done` である．この配線境界は単なるソフトウェア上の規約ではなく，RTL のポート構成として観測可能情報を限定している．

13.7 クロックと通信の安全性

クロックゲーティングをせず `run_enable` で処理 FSM だけを止める．また JTAG 側は `received_toggle`，システム側は 3 段同期 `post_sync` を用いる．結果は `result_snapshot` に保持し，ACK まで上書きしない．これらは FPGA で問題になりやすいクロックグリッチ，メタステブル，データ消失を避けるための回路上の工夫である．

第 14 章

制約，課題，今後の改善

14.1 アルゴリズム上の制約

現行ソルバは局所制約，R4 全体制約，R5 差分制約，最大 12 変数の成分列挙を組み合わせる．12 変数を超えるフロンティア成分は完全列挙できず，分割不能または不完全として閉じる場合がある．また複数成分を残し地雷数で結合するグローバル条件付けは，トップのパラメータとして存在するものの提出設定では有効化していない．このため情報理論的に安全なセルが存在しても，実装上是確定できない可能性がある．

14.2 ヒューリスティックの影響

確定セルがないときは局所成分の確率候補を比較し，初期角選択や低スコア時の辺中央救済を用いる．これは有限時間で進むための実用的な戦略であるが，最適な次手を保証しない．確率の等しい候補間のタイブレークや角・辺中央の順番によって結果が変わる．

14.3 固定容量の課題

`solver_constraint_cache`，`solver_derived_constraint_store`，FIFO，ハッシュの容量とプローブ回数は固定である．大規模盤面や高密度の制約では重複スキップ，閉じた成分，不完全成分が増える可能性がある．改善策として，制約を外部 RAM へ退避する方式，成分サイズに応じた段階的列挙，複数成分を並列処理する方式が考えられる．

14.4 通信・性能の課題

Virtual JTAG メールボックスは 1 コマンド保持型であり，ホストは busy 解除を待って送信する必要がある．1000 盤評価では FPGA 内部の平均処理は約 4.06 ms だが，JTAG 転送と Tcl ポーリングを含めた壁時計時間は別に測定する必要がある．将来はコマンド FIFO，CRC 付きバースト，結果の複数エントリ FIFO を導入すれば通信待ちを短縮できる．

14.5 評価の拡張

今後は Quartus の Fitter レポートから ALM, レジスタ, RAM, Fmax, 電力を取得し, スコアだけでなく資源効率も報告するべきである. また盤面サイズ・地雷密度別に結果を分け, `profile_r123`, `profile_r4`, `profile_r5`, `profile_enumeration` と成功率を対応付けると, どの回路が性能を支配するかを定量化できる.

14.6 保守性

提出 RTL には比較用の four-corners 系や未接続のデバウンサも含まれる. ビルド対象と実験対象を明示的に分け, トップから到達可能なモジュール, 検証専用モジュール, 旧実装をディレクトリまたは QSF で区別すると, 講義資料としての再現性が向上する.

第 15 章

まとめ

本報告では、 19×19 までのマインスイーパを Intel Cyclone IV E FPGA 上で処理するシステムを示した。システムは、完全盤面を保持して開示を行う `minesweeper_game_core`、開示情報だけから推論する決定論ソルバ、メトリクス・表示回路、Virtual JTAG 通信回路に分割されている。

アルゴリズム面では、数字セルの局所制約、盤面全体の残り地雷制約、部分集合差分、制約成分の限定全列挙、確率候補選択を組み合わせた。ハードウェア面では、FSM による逐次化、同期 RAM の待ち状態、固定容量 FIFO と bitmap、乗算器を避けた 19 倍アドレス計算、交差積による確率比較、32 クロックの restoring divider、double-dabble 表示変換を用いた。

通信面では、64 ビット JTAG シフタ、トグル同期型 CDC メールボックス、`jtag_protocol_engine` の `EX_IDLE/BEGIN/DATA/COMMIT`、`board_stream_controller` の `ST_IDLE/CFG/LOAD/COMMIT/RUN` を実装した。CRC と結果 ACK を備えることで、盤面転送・計算・結果取得を分離し、通信エラーとソルバの stalled を区別できる。

実機の speed10 1,000 盤試験では、862 盤を完全解答し、平均スコア 0.9038487、平均 81,110.615 サイクルを得た。これは RTL を FPGA へ移植しただけでなく、入力プロトコル、逐次アルゴリズム、結果評価、表示、PC 側検証を一つの再現可能な実験系としてまとめた成果である。

一方、最大 12 変数の列挙上限、固定容量の制約ストア、グローバル条件付けの未使用、確率選択のヒューリスティックは今後の課題である。これらを拡張すれば完全解答率を高められるが、RAM・論理資源・処理時間とのトレードオフが生じる。FPGA 実装では、アルゴリズムの正しさだけでなく、観測可能情報、レジスタ幅、クロック当たりの処理量、通信の保持期間まで同時に設計する必要があることが、本成果物から得られる主要な知見である。

付録 A

モジュール一覧

表 A.1: RTL モジュールと主な役割

モジュール	主な役割
minesweeper_jtag_submission_local_top	提出用パラメータを固定する最上位ラッパ
minesweeper_jtag_top	JTAG/ROM 入力, 演算系, 結果保持を接続するトップ
minesweeper_mu500_system	ソルバ, 評価, 速度制御, 表示を統合
minesweeper_deterministic_system	ゲームコアと決定論的ソルバを接続
minesweeper_game_core	完全盤面の保持, 選択受付, 0 セルの BFS 連鎖開示
minesweeper_solver_deterministic	開示情報だけを用いる推論ソルバ本体
solver_state_ram	セル状態と開示済み手掛かりの同期 RAM
solver_safe_fifo	確定安全セルの重複抑止付き FIFO
solver_dirty_fifo	再評価対象セルの重複抑止付き FIFO
solver_neighbor_scan	数字セルの 8 近傍走査と局所推論
solver_constraint_cache	数字セルごとの局所制約キャッシュ
solver_r5_compare	二制約の部分集合差分計算
solver_derived_constraint_store	正規化した派生制約のハッシュ表
solver_component_adapter	局所マスクをセル番号ストリームへ変換
solver_component_builder	最大 12 変数・16 制約の連結成分を構築
solver_small_enumerator	制約を満たす割当の深さ優先完全列挙
solver_forced_cell_walker	確定変数を盤面セル番号へ逆変換
solver_small_component_pipeline	変換, 構築, 列挙, 結果出力を統括
board_stream_controller	密な行優先入力を固定ストライド 19 へ変換
jtag_protocol_engine	64 bit ホストコマンドの解釈と応答生成
virtual_jtag_shift64	JTAG クロック領域の 64 bit シフトレジスタ
virtual_jtag_mailbox	JTAG と 20 MHz 領域間の 1 コマンド CDC
result_snapshot	ACK まで 1 盤面分の結果を保持
board_rom_loader	3 枚の小規模スモークテスト盤を供給
minesweeper_metrics	盤面別・累積スコアと回数を計算
unsigned_divider	32 サイクルの符号なし復元除算器
minesweeper_display_formatter	数値を BCD および 7 セグメントパターンへ変換
mu500_7seg_latch_driver	64 桁と 64 LED のラッチ選択信号を駆動
solver_speed_control	FSM 用 clock enable と heartbeat を生成
minesweeper_solver_four_corners	比較用の 4 隅選択ソルバ
minesweeper_four_corners_system	比較用ソルバとゲームコアを接続
button_press_debouncer	機械式ボタンの同期化とデバウンス (提出トップでは未使用)

A.1 提出構成と比較用構成

提出用 QSF は共通設定 `minesweeper_mu500.qsf` を読み込んだ後, トップレベルを `minesweeper_jtag_submission_local_top` へ上書きする. したがって, QSF に列挙される全モジュールが提出時の実行経路に存在するとは限らない. 特に `minesweeper_solver_four_corners` 系は比較用であり, 提出ラッパは `DETERMINISTIC_SOLVER=1` を指定する.

付録 B

ビルドおよび実行手順

B.1 RTL シミュレーション

リポジトリルートから次の PowerShell コマンドを実行する.

```
.\fpga\sim\run_official_file.ps1 `
-BoardFile .\fpga\official\board_pack_1000_fpga.txt `
-ResultCsv fpga\simulation_1000.csv `
-MaxBoards 1000 `
-MaxProbabilityGuesses 15 `
-BaseProbabilityGuesses 8 `
-SaturatingHalfSafeFeedback `
-LowScoreEdgeRescue `
-LowScoreRescueSafeThreshold 16 `
-Deterministic
```

B.2 Quartus コンパイル

Quartus Prime Lite 24.1std を用い, 次を実行する.

```
quartus_sh --flow compile `
.\fpga\quartus\minesweeper_submission_local_mu500
```

制約ファイル minesweeper_mu500.sdc では CLK_20MHZ に周期 50 ns, すなわち 20 MHz のクロックを定義している.

B.3 実機書込みと 1000 盤試験

```
quartus_pgm -m jtag -o `
"p:fpga\deliverables\minesweeper_submission_local_mu500.sof@1"
```

```
quartus_stp -t fpga/sim/jtag_official_runner.tcl `
```

```
fpga/official/board_pack_1000_fpga.txt `
fpga/hardware_1000.csv 1000 0 fast
```

B.4 本文の生成

本報告書は LuaLaTeX を前提とする. TeX Live 環境で `fpga/report` へ移動し, 次を実行する.

```
latexmk -lualatex main.tex
```

参考文献

- [1] Intel Corporation. *Cyclone IV Device Handbook*. Intel Corporation.
- [2] IEEE. *IEEE Standard for Verilog Hardware Description Language*, 2006.
- [3] David Money Harris and Sarah L. Harris. *Digital Design and Computer Architecture*. Morgan Kaufmann.